

---

# **Aliro\_Actuator**

**Mike Jongen**

**May 21, 2024**



**MODULES**

**1 Approach 1 (handle) 3**

**2 Approach 2 (command/response) 5**

**3 Modules 7**

3.1 aliro\_actuator . . . . . 7

3.1.1 aliro\_actuator package . . . . . 7

3.1.1.1 Subpackages . . . . . 7

3.1.1.2 Submodules . . . . . 43

3.1.1.3 Module contents . . . . . 44

**4 Indices and tables 45**

**Python Module Index 47**

**Index 49**



The code is separated into 5 parts: the 4 parts as described in the Architecture overview of the specification (chapter 5.1) and the hardware drivers.

This actuator supports both the reader side and the user side of the protocol. The class that implements the reader side is the *Reader*, and the user side is implemented by the *UserDevice*.

Both *Reader* and *UserDevice* have a similar structure. They both have a parameter `transport_protocol` (of type *TransportProtocol*) which indicates which transport protocol to use (NFC or BLE/UWB). After instantiating the class, the `transaction_initiation()` or `transaction_initiation()` method is used to set up the connection to a device. This method blocks until the connection is established.

The *UserDevice* and *Reader* can now be used in one of two ways: using the handle functions for an easy to use approach, or the command/response methods for more control.



## APPROACH 1 (HANDLE)

This approach uses the attributes set during instantiation, and the ones received from commands, to make the actuator easier to use. Start by using the `start_new_session()` or `start_new_session()` to start a session. All data received will be stored in this session. The *UserDevice* now needs to wait for a command. This is done with the `wait_for_command()` method, which returns a *Command* to pass to a handle method. Now, use the methods that start with 'handle' to handle the commands:

User methods:

```
handle_select()
handle_auth0()
handle_auth1()
handle_control_flow()
handle_exchange()
handle_load_cert()
```

Reader methods:

```
handle_select()
handle_auth0()
handle_auth1()
handle_control_flow()
handle_exchange()
handle_load_cert()
```



## **APPROACH 2 (COMMAND/RESPONSE)**

This approach gives more control to the programmer. The methods that start with ‘command’/‘response’ require a value for every piece of data they will send. The data received will also need to be manually processed by the programmer (unlike Approach 1, there is no session to save received data). Note that Approach 1 also uses these ‘command’/‘response’ methods, so you can use the Approach 1 methods as an example.



## MODULES

### 3.1 aliro\_actuator

#### 3.1.1 aliro\_actuator package

##### 3.1.1.1 Subpackages

**aliro\_actuator.access\_document package**

##### Submodules

**aliro\_actuator.access\_document.access\_credential module**

**class** aliro\_actuator.access\_document.access\_credential.AccessDocument

Bases: *Document*

**parse()** → None

**validate\_credential()** → None

**class** aliro\_actuator.access\_document.access\_credential.DataElement(*id: bytes | None, schedules: list | None, access\_rules: list | None, reader\_rules\_ids: list | None, validity\_iteration: int | None*)

Bases: object

**class** aliro\_actuator.access\_document.access\_credential.IssuerAuth

Bases: object

**aliro\_actuator.access\_document.errors module**

**exception** aliro\_actuator.access\_document.errors.DocumentError

Bases: Exception

**exception** aliro\_actuator.access\_document.errors.ElementSizeError(*actual: int, maximum: int | None = None, expected: int | None = None, minimum: int | None = None*)

Bases: *DocumentError*

**aliro\_actuator.access\_document.revocation\_document module**

**class** aliro\_actuator.access\_document.revocation\_document.**RevocationDocument**

Bases: *Document*

**Module contents**

**class** aliro\_actuator.access\_document.**Document**

Bases: object

**get\_timestamp()** → bytes | None

**retrieve()** → None

**store()** → None

**aliro\_actuator.access\_protocol package****Submodules****aliro\_actuator.access\_protocol.apdu module**

**class** aliro\_actuator.access\_protocol.apdu.**APDU**

Bases: object

Class for creating and parsing APDU messages.

Use the `parse_command/response` functions to parse the received commands/responses. Use the `create_<cmd>_command/response` to create new command/responses. The `as_bytes` attribute of these commands/responses gives a bytestring which can be send.

has the following attributes: `support_extended_length_apdu`: bool (set to true to support extended length APDU's, see 8.3 Aliro spec)

**create\_auth0\_command**(*transaction\_type*: *Transaction*, *transaction\_code*: *TransactionCode*,  
*protocol\_version*: int, *reader\_epubk*: bytes, *transaction\_identifier*: bytes,  
*reader\_identifier*: bytes, *vendor\_extension*: bytes | None = None) → *Command*

**create\_auth0\_response**(*credential\_epubk*: bytes, *status*: int, *cryptogram*: bytes | None = None) →  
*Response*

**create\_auth1\_command**(*response*: *Auth1Response*, *reader\_sig*: bytes, *certificate\_data*: bytes | None =  
None) → *Command*

**create\_auth1\_response**(*key\_slot*: bytes | None, *public\_key*: bytes | None, *expected\_response*:  
*Auth1Response*, *signature*: bytes, *encryption*: *EncryptionEngine*, *status*: int =  
*StatusBytes.SUCCESS*, *private\_mailbox\_data*: bytes | None = None,  
*signaling\_bitmap*: bytes | None = None, *credential\_signed\_timestamp*: bytes |  
None = None, *revocation\_signed\_timestamp*: bytes | None = None) → *Response*

**create\_command**(*cla*: int, *ins*: int, *p1*: int, *p2*: int, *data*: bytes, *le*: int | None) → *Command*

Create a command. (the other more specific functions are recommended)

```

create_control_flow_command(S1: int, S2: int, domain_specific_data: bytes | None = None) →
    Command

create_control_flow_response(status: int) → Response

create_envelope_command(payload: bytes) → Command

create_envelope_response(payload: bytes | None = None, status: int = StatusBytes.SUCCESS) →
    Response

create_error_response(status_bytes: int) → Response

create_exchange_command(atomic_session: bool, payload_tlv: TLV, encryption: EncryptionEngine) →
    Command

create_exchange_response(payload: bytes, encryption: EncryptionEngine, status: int) → Response

create_get_response_command(payload: bytes) → Command

create_get_response_response(payload: bytes, status: int) → Response

create_load_cert_command(compressed_reader_cert: bytes) → Command

create_load_cert_response(status: int) → Response

create_select_command(aid: bytes) → Command

create_select_response(AID: bytes, type: int, expedited_phase_supported_protocol_versions: list[int],
    maximum_command_apdu: int | None = None, maximum_response_apdu: int |
    None = None, vendor_specific_tlv: TLV | None = None, status: int =
    StatusBytes.SUCCESS) → Response

parse_command(command_as_bytes: bytes, encryption: EncryptionEngine | None = None) → Command
    Parse a command bytestring. Used to extract info from a received command.

parse_response(response_as_bytes: bytes, ins: INS, encryption: EncryptionEngine | None = None) →
    Response
    Parse a response bytestring. Used to extract info from a received response.

class aliro_actuator.access_protocol.apdu.Auth1Response(value)
    Bases: IntEnum

    Indicating the type of response requested in a auth1 command. Send with tag 0x41, bit 0. See table 8-10 of the
    Aliro spec.

    CREDENTIAL_PUBLIC_KEY = 1

    KEY_SLOT = 0

class aliro_actuator.access_protocol.apdu.Command
    Bases: Message

    contains an APDU command.

    has the following attributes: as_bytes: bytes (full command as bytes) cla: int ins: int p1: int p2: int lc: int data:
    bytes le: int

    classmethod create_from_bytestring(bytestring: bytes) → Command
        Create a Command from a bytestring.

```

**classmethod** **create\_from\_parameters**(*cla: int, ins: int, p1: int, p2: int, data: bytes, le: int | None*) → *Command*

Create a Command from its fields.

lc is calculated from the data. Set le to None if no le field is required.

**parse\_as\_auth0**() → None

Parse this command as a AUTH0 command.

Checks the fields and raises errors for invalid fields. creates the following attributes: `command_parameters: int transaction_code: int expedited_phase_protocol_version: int reader_epubk: bytes transaction_identifier: bytes reader_identifier: bytes vendor_specific_extension: bytes | None`

**parse\_as\_auth1**() → None

Parse this command as a Auth1 command.

Checks the fields and raises errors for invalid fields. creates the following attributes: `command_parameters: int expected_response: Auth1Response request_access_credentials: bool reader_signature: bytes certificate_data: bytes | None`

**parse\_as\_control\_flow**() → None

Parse this command as a Control Flow command.

Checks the fields and raises errors for invalid fields.

**parse\_as\_envelope**() → None

Parse this command as a Envelope command.

Checks the fields and raises errors for invalid fields.

**parse\_as\_exchange**(*encryption: EncryptionEngine | None = None*) → None

Parse this command as a Exchange command.

Checks the fields and raises errors for invalid fields. creates the following attributes: `encrypted_payload authentication_tag decrypted_payload atomic_session payload_tlv read_requests write_requests set_requests notify ursk update_doc`

**parse\_as\_get\_response**() → None

Parse this command as a Get Response command.

Checks the fields and raises errors for invalid fields.

**parse\_as\_load\_cert**() → None

Parse this command as a Load Cert command.

Checks the fields and raises errors for invalid fields.

**parse\_as\_select**() → None

Parse this command as a Select command.

Checks the fields and raises errors for invalid fields.

**class** **aliro\_actuator.access\_protocol.apdu.INS**(*value*)

Bases: `IntEnum`

Possible values of the INS field in an APDU message. See Table 8-2 of of the Aliro spec.

**AUTH0** = 128

**AUTH1** = 129

**CONTROL\_FLOW = 60**

**ENVELOPE = 195**

**EXCHANGE = 201**

**GET\_RESPONSE = 192**

**LOAD\_CERT = 209**

**SELECT = 164**

**class aliro\_actuator.access\_protocol.apdu.Message**

Bases: object

Parent class of Command and Response. Contains functions that are common for both Command and Response.

**to\_bytes()** → bytes

**class aliro\_actuator.access\_protocol.apdu.Response**

Bases: *Message*

contains an APDU Response.

has the following attributes: as\_bytes: bytes (full command as bytes) sw1: int sw2: int status: int (combination of sw1 and sw2) data: bytes

**classmethod create\_from\_bytestring**(bytestring: bytes) → *Response*

Create a Response from a bytestring.

**classmethod create\_from\_parameters**(data: bytes | None = None, status: int = StatusBytes.SUCCESS) → *Response*

Create a Response from its fields.

**parse\_as\_auth0**() → None

Parse this response as a Auth0 response.

creates the following attributes: credential\_epubk: bytes cryptogram: bytes (if present) vendor\_specific\_extensions: tlv (if present)

**parse\_as\_auth1**(encryption: *EncryptionEngine* | None = None) → None

Parse this response as a Auth1 response.

creates the following attributes: encrypted\_payload: bytes authentication\_tag: bytes decrypted\_payload: bytes payload\_tlv: TLV key\_slot: bytes | None credential\_public\_key: bytes | None user\_device\_signature: bytes private\_mailbox\_data: bytes | None signaling\_bitmap: bytes credential\_signed\_timestamp: bytes | None revocation\_signed\_timestamp: bytes | None

**parse\_as\_control\_flow**() → None

Parse this response as a control\_flow response.

**parse\_as\_envelope**() → None

Parse this response as a envelope response.

**parse\_as\_exchange**(encryption: *EncryptionEngine* | None = None) → None

Parse this response as a Exchange response.

creates the following attributes: encrypted\_payload: bytes authentication\_tag: bytes decrypted\_payload: bytes status\_code: bytes read\_data: bytes

**parse\_as\_get\_response()** → None

Parse this response as a get\_response response.

**parse\_as\_load\_cert()** → None

Parse this response as a Auth1 response.

**parse\_as\_select()** → None

Parse this response as a Select response.

creates the following attributes: compl\_aid: bytes proprietary\_tlv: TLV type: bytes expedited\_phase\_supported\_protocol\_versions: bytes maximum\_command\_apdu: int maximum\_response\_apdu: int vendor\_specific\_extensions: TLV

**property sw1: int**

**property sw2: int**

**class aliro\_actuator.access\_protocol.apdu.S1(value)**

Bases: IntEnum

Indicating the S1 parameter in a control flow command. Send with tag 0x41. See table 8-13 and 8.5.7.2 of the Aliro spec.

**FINISHED\_WITH\_FAILURE = 0**

**FINISHED\_WITH\_SUCCESS = 1**

**class aliro\_actuator.access\_protocol.apdu.S2(value)**

Bases: IntEnum

Indicating the S2 parameter in a control flow command. Send with tag 0x42. See table 8-13 and 8.5.7.3 of the Aliro spec.

**NONE = 0**

**PROTOCOL\_VERSION\_NOT\_SUPPORTED = 39**

**class aliro\_actuator.access\_protocol.apdu.StatusBytes(value)**

Bases: IntEnum

Indicating (some) known values of the status bytes returned in a response.

**COMMAND\_NOT\_ALLOWED = 26880**

**COMMAND\_NOT\_COMPLIANT = 26369**

**CONDITIONS\_OF\_USE\_NOT\_SATISFIED = 27013**

**FILE\_OR\_APP\_NOT\_FOUND = 27266**

**FUNCTIONS\_IN\_CLA\_NOT\_SUPPORTED = 26624**

**FUNCTION\_NOT\_SUPPORTED = 27265**

**GENERIC\_ERROR = 25600**

**INCORRECT\_P1\_P2 = 27392**

**INCORRECT\_PARAMETERS\_IN\_DATA = 27264**

**INCORRECT\_SECURE\_MESSAGING\_DOS = 27016**

```

INVALID_CLASS = 28160
INVALID_INSTRUCTION = 27904
LOGICAL_CHANNEL_NOT_SUPPORTED = 26753
MEMORY_FAILURE = 25985
NO_PRECISE_DIAGNOSIS = 28160
SECURITY_STATUS_NOT_SATISFIED = 27010
SUCCESS = 36864
WRONG_LENGTH_IN_LC = 26368

```

```
class aliro_actuator.access_protocol.apdu.Transaction(value)
```

Bases: IntEnum

Indicating the transaction type in a auth0 command. See table 8-5 of the Aliro spec.

```
FAST = 1
```

```
STANDARD = 0
```

```
class aliro_actuator.access_protocol.apdu.TransactionCode(value)
```

Bases: IntEnum

Indicating the transaction code in a auth0 command. See table 8-1 and 8-3 of the Aliro spec.

```
FORCE_USER_AUTHENTICATION = 3
```

```
USER_DEVICE = 1
```

```
USER_DEVICE_SECURE_ACTION = 2
```

### aliro\_actuator.access\_protocol.authentication module

```
aliro_actuator.access_protocol.authentication.create_reader_authentication(reader_identifier:
                                                                            bytes,
                                                                            credential_epubk:
                                                                            PublicKey,
                                                                            reader_epubk:
                                                                            PublicKey, trans-
                                                                            action_identifier:
                                                                            bytes) → TLV
```

```
aliro_actuator.access_protocol.authentication.create_user_device_authentication(reader_identifier:
                                                                                bytes,
                                                                                creden-
                                                                                tial_epubk:
                                                                                PublicKey,
                                                                                reader_epubk:
                                                                                PublicKey,
                                                                                transac-
                                                                                tion_identifier:
                                                                                bytes) →
                                                                                TLV
```

**aliro\_actuator.access\_protocol.defines module**

```
class aliro_actuator.access_protocol.defines.Auth0
```

```
    Bases: object
```

```
    COMMAND_LEN = 1
```

```
    COMMAND_TAG = 65
```

```
    CREDENTIAL_EPUBK_LEN = 65
```

```
    CREDENTIAL_EPUBK_TAG = 134
```

```
    CRYPTOGRAM_LEN = 64
```

```
    CRYPTOGRAM_TAG = 157
```

```
    ETPV_LEN = 2
```

```
    ETPV_TAG = 92
```

```
    READER_EPUBK_LEN = 65
```

```
    READER_EPUBK_TAG = 135
```

```
    READER_IDENTIFIER_LEN = 32
```

```
    READER_IDENTIFIER_TAG = 77
```

```
    RE_VENDOR_SPECIFIC_MAX_LEN = 128
```

```
    RE_VENDOR_SPECIFIC_TAG = 178
```

```
    TRANSACTION_CODE_LEN = 1
```

```
    TRANSACTION_CODE_TAG = 66
```

```
    TRANSACTION_ID_LEN = 16
```

```
    TRANSACTION_ID_TAG = 76
```

```
    VENDOR_SPECIFIC_MAX_LEN = 128
```

```
    VENDOR_SPECIFIC_TAG = 177
```

```
class aliro_actuator.access_protocol.defines.Auth1
```

```
    Bases: object
```

```
    CERTIFICATE_TAG = 144
```

```
    COMMAND_LEN = 1
```

```
    COMMAND_TAG = 65
```

```
    CREDENTIAL_PUBK_LEN = 65
```

```
    CREDENTIAL_PUBK_TAG = 90
```

```
    CREDENTIAL_TIMESTAMP_LEN = 20
```

```
CREDENTIAL_TIMESTAMP_TAG = 145

KEY_SLOT_LEN = 8

KEY_SLOT_TAG = 78

MAILBOX_DATA_TAG = 75

READER_SIG_LEN = 64

READER_SIG_TAG = 158

REVOCATION_TIMESTAMP_LEN = 20

REVOCATION_TIMESTAMP_TAG = 146

SIGNALING_BITMAP_LEN = 2

SIGNALING_BITMAP_TAG = 94

USER_DEVICE_SIG_LEN = 64

USER_DEVICE_SIG_TAG = 158

class aliro_actuator.access_protocol.defines.ControlFlow
    Bases: object
    DOMAIN_SPECIFIC_TAG = 67

    S1_LEN = 1

    S1_TAG = 65

    S2_LEN = 1

    S2_TAG = 66

class aliro_actuator.access_protocol.defines.Exchange
    Bases: object
    NOTIFY_TAG = 174

    READ_LEN = 4

    READ_TAG = 135

    SET_LEN = 5

    SET_TAG = 149

    UPDATE_DOC_TAG = 129

    URSK_TAG = 152

    WRITE_TAG = 138

class aliro_actuator.access_protocol.defines.ReaderAuth
    Bases: object
    CREDENTIAL_EPUBK_TAG = 134
```

```
READER_EPUBK_TAG = 135

READER_IDENTIFIER_TAG = 77

TRANSACTION_IDENTIFIER_TAG = 76

USAGE = b'A]\x95i'

USAGE_TAG = 147

class aliro_actuator.access_protocol.defines.Select
    Bases: object
    AID_LEN = 9
    AID_TAG = 132
    ETSPV_TAG = 92
    EXTENDED_INFO_LEN = 8
    EXTENDED_INFO_TAG = 32614
    FCI_TAG = 111
    MAX_COMMAND_LEN = 2
    MAX_COMMAND_TAG = 2
    MAX_RESPONSE_LEN = 2
    MAX_RESPONSE_TAG = 2
    PROPRIETARY_TAG = 165
    TYPE_LEN = 2
    TYPE_TAG = 128
    VENDOR_SPECIFIC_TAG = 179

class aliro_actuator.access_protocol.defines.TransportProtocol(value)
    Bases: IntEnum
    An enumeration.
    BLE_UWB = 1
    NFC = 0
    SOCKET_BLE = 3
    SOCKET_NFC = 2

class aliro_actuator.access_protocol.defines.UserDeviceAuth
    Bases: object
    CREDENTIAL_EPUBK_TAG = 134
    READER_EPUBK_TAG = 135
```

```

READER_IDENTIFIER_TAG = 77

TRANSACTION_IDENTIFIER_TAG = 76

USAGE = b'N\x88{L'

USAGE_TAG = 147

```

### aliro\_actuator.access\_protocol.encryption module

**class** aliro\_actuator.access\_protocol.encryption.**DeviceType**(*value*)

Bases: Enum

Enumerator, used by the EncryptionEngine, to indicate if the device is a reader or user.

**READER** = 1

**USER** = 2

**class** aliro\_actuator.access\_protocol.encryption.**EncryptionEngine**(*devicetype: DeviceType,*  
*exchange\_SK\_reader: bytes,*  
*exchange\_SK\_device: bytes*)

Bases: object

class to encrypt and decrypt messages. Uses AES-256.

**Args:**

devicetype (DeviceType): indicates if the device creating this class is a user or reader  
exchange\_SK\_reader (bytes): exchange SK Reader as described in Figure 8-13 of the Aliro spec  
exchange\_SK\_device (bytes): exchange SK Device as described in Figure 8-13 of the Aliro spec

**check\_counters\_valid()** → bool

Checks the counters as required for the exchange command (Figure 8-15 of the Aliro spec)

**Returns:**

bool: True if the counters are valid (< 0xFFFF), else False

**decrypt**(*ciphertext: bytes, authentication\_tag: bytes, ad: bytes = b''*) → bytes

decrypts encrypted data and indicates if verification succeeded

(figure 8-10 of the Aliro spec)

**Args:**

ciphertext (bytes): Ciphertext. Can be any size. authentication\_tag (bytes): Authentication Tag. Must be 16 bytes. ad (bytes, optional): Additional data. Defaults to b''.

**Raises:**

VerificationError: raised when the verification fails.

**Returns:**

bytes: decrypted plaintext

**encrypt**(*data: bytes, ad: bytes = b''*) → tuple[bytes, bytes]

Generates encrypted data + authentication tag

(figure 8-9 of the Aliro spec)

**Args:**

data (bytes): Data to encrypt. Can be any size. ad (bytes, optional): Additional data. Defaults to b''.

**Returns:**

tuple[bytes, bytes]: tuple of the encrypted data and the authentication tag.

**exception** `aliro_actuator.access_protocol.encryption.VerificationError`

Bases: `AccessProtocolError`

Raised when the verification of an AES decryption fails.

`aliro_actuator.access_protocol.encryption.compute_cryptogram`(*cryptogram\_sk*: bytes,  
*signaling\_bitmap*: bytes,  
*credential\_signed\_timestamp*: bytes  
| *None* = *None*,  
*revocation\_signed\_timestamp*: bytes  
| *None* = *None*) → bytes

`aliro_actuator.access_protocol.encryption.create_proprietary_information`(*type*: int, *expedited\_phase\_supported\_protocol\_versions*: list[int], *maximum\_command\_apdu*: int | *None* = *None*, *maximum\_response\_apdu*: int | *None* = *None*, *vendor\_specific\_tlv*: TLV | *None* = *None*) → TLV

Creates the proprietary information (see Table 10-2), used for salt generation and Select command.

**Args:**

*type* (int): Application type (see Table 10-3) *expedited\_phase\_supported\_protocol\_versions* (list[int]): List of supported protocol versions. *maximum\_command\_apdu* (int | *None*, optional): Defaults to *None*. *maximum\_response\_apdu* (int | *None*, optional): Defaults to *None*. *vendor\_specific\_tlv* (TLV | *None*, optional): Defaults to *None*.

**Returns:**

TLV: Proprietary information (as TLV)

`aliro_actuator.access_protocol.encryption.create_salt`(*transport\_protocol*: TransportProtocol, *word*: bytes, *reader\_public\_key*: PublicKey, *reader\_ephemeral\_public\_key*: PublicKey, *reader\_identifier*: bytes, *protocol\_version*: bytes, *transaction\_identifier*: bytes, *flag*: bytes, *proprietary\_information*: bytes, *credential\_ephemeral\_public\_key*: PublicKey | *None* = *None*) → bytes

Generates the salt used for key generation

**Args:**

*transport\_protocol* (TransportProtocol): The transport protocol used. *word* (bytes): “VolatileFast”, “Volatile\*\*\*\*” or “Persistent\*\*\*\*”. *reader\_public\_key* (PublicKey). *reader\_ephemeral\_public\_key* (PublicKey). *reader\_identifier* (bytes). *protocol\_version* (bytes). *transaction\_identifier* (bytes). *flag* (bytes): *command\_parameters* || *transaction\_code*. *proprietary\_information* (bytes): proprietary information. *credential\_ephemeral\_public\_key* (PublicKey | *None*, optional): only for “VolatileFast” or “Persistent\*\*\*\*”. Defaults to *None*.

**Returns:**

bytes: the salt as bytes

---

```
aliro_actuator.access_protocol.encryption.decrypt_cryptogram(cryptogram_sk: bytes, cryptogram:
                                                            bytes, authentication_tag: bytes) →
                                                            bytes
```

### aliro\_actuator.access\_protocol.errors module

```
exception aliro_actuator.access_protocol.errors.AccessProtocolError
```

Bases: `Exception`

Parent class of all errors related to the access protocol.

```
exception aliro_actuator.access_protocol.errors.CreateCommandError
```

Bases: `AccessProtocolError`

```
exception aliro_actuator.access_protocol.errors.CreateResponseError
```

Bases: `AccessProtocolError`

```
exception aliro_actuator.access_protocol.errors.CryptogramNotFound
```

Bases: `AccessProtocolError`

```
exception aliro_actuator.access_protocol.errors.InvalidAIDError(command: bytes, aid: bytes)
```

Bases: `InvalidCommandDataError`

Raised when a command with an invalid AID is received.

```
exception aliro_actuator.access_protocol.errors.InvalidCLAError(command: bytes)
```

Bases: `InvalidCommandError`

Raised when the received command has an invalid CLA.

```
exception aliro_actuator.access_protocol.errors.InvalidCommandDataError(command: bytes,
                                                                           message: str | None =
                                                                           None)
```

Bases: `InvalidCommandError`

Raised when the received command has an invalid Data field.

```
exception aliro_actuator.access_protocol.errors.InvalidCommandError(command: bytes)
```

Bases: `AccessProtocolError`

Raised when the received command is invalid.

```
exception aliro_actuator.access_protocol.errors.InvalidINSerror(command: bytes)
```

Bases: `InvalidCommandError`

Raised when the received command has an invalid INS.

```
exception aliro_actuator.access_protocol.errors.InvalidLcError(command: bytes)
```

Bases: `InvalidCommandError`

Raised when the received command has an invalid LC.

```
exception aliro_actuator.access_protocol.errors.InvalidLeError(command: bytes)
```

Bases: `InvalidCommandError`

Raised when the received command has an invalid Le.

**exception** `aliro_actuator.access_protocol.errors.InvalidParameterError`(*command: bytes*)

Bases: [`InvalidCommandError`](#)

Raised when the received command has an invalid Parameter (P1 or P2).

**exception** `aliro_actuator.access_protocol.errors.InvalidResponseDataError`(*response: bytes,*  
*other\_info: str = ""*)

Bases: [`InvalidResponseError`](#)

Raised when the received Response has an invalid data.

**exception** `aliro_actuator.access_protocol.errors.InvalidResponseError`(*response: bytes*)

Bases: [`AccessProtocolError`](#)

Raised when the received response is invalid.

**exception** `aliro_actuator.access_protocol.errors.InvalidStatusError`(*response: bytes, status: int*)

Bases: [`InvalidResponseError`](#)

Raised when the received Response has an invalid status.

**exception** `aliro_actuator.access_protocol.errors.MessageTooLongError`

Bases: [`AccessProtocolError`](#)

Raised when the received command is too long. See 8.3 of the Aliro Spec.

**exception** `aliro_actuator.access_protocol.errors.SessionError`

Bases: [`AccessProtocolError`](#)

**exception** `aliro_actuator.access_protocol.errors.UnexpectedCommandError`

Bases: [`AccessProtocolError`](#)

Raised when a different command is received than expected.

**exception** `aliro_actuator.access_protocol.errors.UnexpectedNotificationDataError`(*response:*  
*bytes,*  
*other\_info:*  
*str = ""*)

Bases: [`AccessProtocolError`](#)

**exception** `aliro_actuator.access_protocol.errors.UnexpectedResponseError`

Bases: [`AccessProtocolError`](#)

Raised when a different Response is received than expected.

**exception** `aliro_actuator.access_protocol.errors.VersionError`

Bases: [`AccessProtocolError`](#)

Raised when the requested version is not supported.

**aliro\_actuator.access\_protocol.mailbox module**

```
class aliro_actuator.access_protocol.mailbox.Mailbox(initial_data: list[tuple[bytes, int, bytes]] | None
                                                    = None, size: int = 0, read_permission: bool =
                                                    True, write_permission: bool = True)
```

Bases: object

**DATA\_TAG** = 130

**INDEX\_TAG** = 129

**START\_OF\_DATA\_TAG** = 96

**check\_boundaries**(offset: int, length: int) → bool

Checks if the offset and length combination is a valid option for this mailbox.

**Args:**

offset (int): Offset in bytes. length (int): Data to write.

**Returns:**

bool: True if offset and length fit in this mailbox, else False

**data\_is\_set**() → bool

Checks if some data is different from zeros

**Returns:**

bool: True if not all data is 0x00, else False

**get\_data**() → bytes

Returns just the mailbox data (no index, tags or lengths).

**Returns:**

bytes: mailbox data, not including tag (0x82) and length.

**get\_raw**() → bytes

Returns the mailbox contents as bytes.

**Returns:**

bytes: mailbox contents (including index and data)

**get\_tlv**() → *TLV*

Returns the mailbox contents as a TLV.

**Returns:**

TLV: Mailbox contents.

**read**(offset: int, length: int) → bytes

Returns a block of data from the mailbox.

**Args:**

offset (int): offset in bytes. length (int): length in bytes.

**Raises:**

MailboxPermissionError: raised if read\_permission is not set.

**Returns:**

bytes: Data from offset to offset + length

**set**(*offset: int, length: int, value: int*) → None

Set a block of data to a single value.

**Args:**

offset (int): offset in bytes. length (int): length in bytes value (int): Every byte from offset to offset + length is set to this value.

**Raises:**

MailboxPermissionError: raised if write\_permission is not set.

**write**(*offset: int, data: bytes*) → None

Write a block of data to the mailbox.

**Args:**

offset (int): Offset in bytes. data (bytes): Data to write.

**Raises:**

MailboxPermissionError: raised if write\_permission is not set.

**exception** aliro\_actuator.access\_protocol.mailbox.**MailboxFormatError**

Bases: [AccessProtocolError](#)

Raised when data passed to the mailbox has the wrong format.

**exception** aliro\_actuator.access\_protocol.mailbox.**MailboxPermissionError**

Bases: [AccessProtocolError](#)

Raised when trying to read/write the mailbox without correct permissions.

## aliro\_actuator.access\_protocol.reader module

**class** aliro\_actuator.access\_protocol.reader.**Reader**(*transport\_protocol: [TransportProtocol](#),  
transport\_override: [TransportProtocolBase](#) |  
None = None, reader\_group\_identifier: bytes |  
None = None, reader\_group\_sub\_identifier: bytes  
| None = None, reader\_cert: bytes | None = None,  
reader\_key: [KeyPair](#) | None = None,  
vendor\_extension: bytes | None = None,  
fast\_transaction\_implemented: bool = True,  
reader\_storage: [ReaderStorage](#) | None = None,  
group\_resolving\_key: bytes =  
b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',  
spsm: bytes = b'\x00\x80',  
transaction\_identifier\_list: list[bytes] | None =  
None, ephemeral\_key\_list: list[[KeyPair](#)] | None =  
None)*)

Bases: [Device](#)

Simulates a reader device.

**Args:**

transport\_protocol ([TransportProtocol](#)): Transport protocol to use. transport\_override ([TransportProtocolBase](#) | None, optional): Override the transport protocol. Mainly used for testing. Defaults to None. reader\_group\_identifier (bytes | None, optional): Part of the reader\_identifier. Defaults to None. reader\_group\_sub\_identifier (bytes | None, optional): Part of the reader\_cert (bytes | None, optional): Reader certificate. Defaults to None. reader\_key ([KeyPair](#) | None, optional): Reader Key. Defaults to None. vendor\_extension (bytes | None, optional): Defaults to None. fast\_transaction\_implemented

(bool): Indicates if this reader implements the fast transaction. Defaults to True. reader\_storage (ReaderStorage | None, optional): Defaults to None. group\_resolving\_key (bytes, optional): Defaults to 0x00000000000000000000000000000000. spsm (bytes, optional): Defaults to 0x0080. transaction\_identifier\_list (list[bytes] | None, optional): list of transaction identifiers to be used by the reader. first transaction uses index 0, second transaction uses index 1, etc. transaction identifiers are randomly generated if this is set to None. Defaults to None. ephemeral\_key\_list (list[KeyPair] | None, optional): list of ephemeral keys to be used by the reader. first transaction uses index 0, second transaction uses index 1, etc. Ephemeral keys are randomly generated if this is set to None. Defaults to None.

**Raises:**

AccessProtocolError: Raised when arguments have invalid format.

**async command\_auth0**(transaction: Transaction, transaction\_code: TransactionCode, protocol\_version: int, reader\_epubk: bytes, transaction\_identifier: bytes, reader\_identifier: bytes, vendor\_extension: bytes | None = None) → Response

Create and send a auth0 command, and wait for a response.

**Args:**

transaction (Transaction): fast or standard transaction\_code (TransactionCode): code with instruction (ex. Lock/Unlock) protocol\_version (int): reader\_epubk (bytes): Reader Ephemeral Key as bytes transaction\_identifier (bytes): reader\_identifier (bytes): vendor\_extension (bytes | None): Vendor specific extension TLV. Defaults to None.

**Returns:**

Response: Response containing the received data.

**async command\_auth1**(expected\_response: Auth1Response, reader\_identifier: bytes, credential\_epubk: PublicKey, reader\_epubk: PublicKey, transaction\_identifier: bytes, encryption: EncryptionEngine | None = None) → Response

Create and send a auth1 command, and wait for a response.

**Args:**

expected\_response (Auth1Response): key slot or credential public key reader\_identifier (bytes): credential\_epubk (PublicKey): reader\_epubk (PublicKey): transaction\_identifier (bytes): encryption (EncryptionEngine | None, optional): Encryption engine to decrypt the response. Response will not be decrypted if this is None. Defaults to None.

**Returns:**

Response: Response containing the received data.

**async command\_control\_flow**(s1: int, s2: int, domain\_specific\_data: bytes | None = None) → Response

Create and send a exchange command, and wait for a response.

**Args:**

s1 (int): s2 (int): domain\_specific\_data (bytes | None, optional): Defaults to None.

**Returns:**

Response: Response containing the received data.

**command\_envelope**() → None

**async command\_exchange**(atomic\_session: bool, payload: TLV, encryption: EncryptionEngine) → Response

Create and send a exchange command, and wait for a response.

**Args:**

atomic\_session (bool): if True, this is part of an atomic session payload (TLV): The payload to send. encryption (EncryptionEngine): Encryption engine to encrypt the message and decode the response.

**Returns:**

Response: Response containing the received data.

**command\_get\_response()** → None

**async command\_load\_cert**(*compressed\_cert: bytes*) → *Response*

Create and send a load\_cert command, and wait for a response.

**Args:**

compressed\_cert (bytes): compressed certificate to send.

**Returns:**

Response: Response containing the received data.

**async command\_select**(*aid: bytes*) → *Response*

Create and send a select command, and wait for a response.

**Args:**

aid (bytes): AID to be send.

**Returns:**

Response: Response containing the received data.

**async decrypt\_cryptogram**(*cryptogram: bytes | None*) → None

**end\_session()** → None

End the current reader session.

**async expedited\_transaction\_fast**(*transaction\_code: TransactionCode*) → None

**async expedited\_transaction\_standard**(*transaction\_code: TransactionCode, load\_cert: bool = False*) → None

Runs the Expedited Standard Phase.

**Args:**

transaction\_code (TransactionCode): Passed during AUTH0. load\_cert (bool, optional): Runs the load\_cert command if True. Defaults to False.

**async failure\_process()** → None

Should be called when a failure state has occurred. Destroys all session bound keys and data. If transport protocol is NFC, a control\_flow command indicating failure is send. If transport protocol is BLE, a failure event message is send.

**async handle\_auth0**(*transaction\_type: Transaction, transaction\_code: TransactionCode*) → None

Create and send a AUTH0 command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

**Args:**

transaction\_type (Transaction): fast or standard transaction\_code (TransactionCode): code with instruction (ex. Lock/Unlock)

**Raises:**

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data.

**async handle\_auth1**(*expected\_response: Auth1Response = Auth1Response.CREDENTIAL\_PUBLIC\_KEY*) → None

Create and send a AUTH1 command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

**Raises:**

SessionError: Raised if no session is found.

**async handle\_auth1\_credential\_public\_key**(*expected\_response: int, credential\_public\_key\_bytes: bytes | None, key\_slot: bytes | None*) → None

**async handle\_control\_flow**(*success: bool*) → None

Create and send a control\_flow command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

**Args:**

success (bool): if True, the command will indicate a success status.

**Raises:**

SessionError: Raised if no session is found.

**async handle\_exchange**(*atomic\_session: bool, read\_requests: list[tuple[int, int]] | None = None, write\_requests: list[tuple[int, bytes]] | None = None, set\_requests: list[tuple[int, int, int]] | None = None, notify: TLV | None = None, ursk: bytes | None = None, update\_doc: bytes | None = None*) → list[bytes]

Create and send a exchange command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

**Args:**

atomic\_session (bool): if True, this is part of an atomic session read\_requests (list[tuple[int, int]] | None): List of (offset, length) tuples. write\_requests (list[tuple[int, bytes]] | None): list of (offset, data) tuples. set\_requests (list[tuple[int, int, int]] | None): list of (offset, length, value) tuples. notify (TLV | None): Notify TLV ursk (bytes | None): URSK, for BLE update\_doc (bytes | None): request update of an existing Access Document

**Raises:**

SessionError: Raised if no session is found.

**Returns:**

list[bytes]: list of read data.

**async handle\_load\_cert**() → None

Create and send a load\_cert command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

**Raises:**

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data.

**async handle\_select**(*aid: bytes*) → None

create and send a select command. Required data from is retrieved from the Reader (self) and the session. The data contained in the response is stored in the session.

**Args:**

aid (bytes): AID to be send.

**Raises:**

SessionError: Raised if no session is found. AccessProtocolError: Raised if the response has invalid data. UnexpectedResponseError: Raised if the response has status/data that cannot be handled

**property reader\_group\_identifier:** bytes

**property reader\_group\_sub\_identifier:** bytes

**property reader\_identifier:** bytes

**async setup\_connection()** → None

Setup up the connection to the User device.

**start\_new\_session()** → None

Start a new reader session. Must be done before using handle commands. This sessions stores all information received from commands. Start a new session to delete all received info and start over.

**step\_up\_transaction()** → None

**async transaction\_initiation()** → None

Initializes the hardware and sets up a connection to the card.

**async transaction\_termination()** → None

Terminates the connection to the user device.

**async wait\_for\_initiate\_access\_protocol\_notification()** → None

**class** aliro\_actuator.access\_protocol.reader.ReaderFastCacheEntry(*access\_credential: PublicKey, kpersistent: bytes*)

Bases: object

**print\_to\_log()** → None

**class** aliro\_actuator.access\_protocol.reader.ReaderSession(*reader\_key: KeyPair, reader\_identifier: bytes, vendor\_extension: bytes | None = None*)

Bases: object

Contains info from a single session (with one User Device)

**can\_retrieve\_access\_credential()** → bool

**can\_retrieve\_revocation\_document()** → bool

**check\_user\_device\_authentication**(*user\_device\_signature: bytes*) → bool

**decrypt\_payload**(*encrypted\_payload: bytes, authentication\_tag: bytes*) → bytes

**derive\_key\_persistent**(*transport\_protocol: TransportProtocol, credential: PublicKey*) → bytes

**derive\_key\_volatile**(*transport\_protocol: TransportProtocol*) → None

**derive\_key\_volatile\_fast**(*transport\_protocol: TransportProtocol, credential: PublicKey, k\_persistent: bytes*) → None

**encrypt\_payload**(*payload: bytes*) → tuple[bytes, bytes]

**generate\_ephemeral\_key**(*ephemeral\_key: KeyPair | None = None*) → None

**get\_credential\_ephemeral\_key()** → bytes

**get\_reader\_epubkey()** → *PublicKey*

**lookup\_credential\_public\_key**(*key\_slot: bytes*) → *PublicKey*

**property reader\_group\_identifier:** bytes

```

property reader_group_sub_identifier: bytes
property reader_identifier: bytes
set_auth1_info(auth1_response: Response) → None
set_credential_ephemeral_key(key: PublicKey) → None
set_credential_public_key(key: PublicKey) → None
set_cryptogram_info(decrypted_cryptogram: TLV) → None
set_flag(transaction: Transaction, transaction_code: TransactionCode) → None
set_initiate_access_protocol_info(initiate_access_protocol_notification: bytes) → None
set_response_vendor_extension(vendor_extension: TLV | None) → None
set_select_info(select_response: Response) → None
set_shared_key() → None
step_up_aid_select_required() → bool
property transaction_identifier: bytes
class aliro_actuator.access_protocol.reader.ReaderStorage
    Bases: object
    Cross-session storage for Expedited Fast cached data
    add_kpersistent(access_credential: PublicKey, kpersistent: bytes) → ReaderFastCacheEntry
    clear_kpersistent() → None
    get_kpersistent_list() → list[ReaderFastCacheEntry]
    remove_kpersistent(access_credential: PublicKey) → None

```

### aliro\_actuator.access\_protocol.tlv module

```

class aliro_actuator.access_protocol.tlv.TLV(data: list[tuple[int, bytes | list]])
    Bases: object
    Class to handle TLV structures (BER-TLV, ISO 7816-4) (use the from_bytes method to create this struct from a
    bytestring)
    Args:
        data (list[tuple[int, bytes | list]]): list of tuples. The tuples consist of tags and values. The values can
        be either bytes or a list of tag/value tuples. (length is calculated using the data and does not need to be
        specified)
    add_value(tag: int, data: bytes) → None
        add a new tag/value pair to the TLV
    Args:
        tag (int): tag data (bytes): value

```

**static from\_bytes**(*data\_bytes: bytes*) → *TLV*

Converts tlv value (BER-TLV, ISO 7816-4) from bytestring to TLV class Value is b"" if the tlv tag has no value. (length is not represented in the class, but can be derived from the value)

**Raises:**

TlvError: error during parsing of the bytestring

**get\_all\_bytes\_of\_tag**(*tag: int*) → list[bytes]

Similar to get\_all\_of\_tag, but the list only contains bytes objects. Raises an error when a no byte object is found.

**Args:**

tag (int): tag number

**Raises:**

TlvError: Raised when value is not of type bytes.

**Returns:**

list[bytes]: list with all values with given tag

**get\_all\_of\_tag**(*tag: int*) → list[bytes | *TLV*]

returns a list with all values with a given tag.

**Args:**

tag (int): tag number

**Returns:**

list[bytes | TLV]: list with all values with given tag. Can be empty if tag was not found.

**get\_all\_tlv\_of\_tag**(*tag: int*) → list[*TLV*]

Similar to get\_all\_of\_tag, but the list only contains tlv objects. Raises an error when a no byte object is found.

**Args:**

tag (int): tag number

**Raises:**

TlvError: Raised when value is not of type bytes.

**Returns:**

list[bytes]: list with all values with given tag

**get\_bytes**(*tag: int, index: int = 0*) → bytes

Similar to get\_value, but always returns a bytestring. Raises error when requested element is not a bytestring

**Args:**

tag (int): tag number index (int, optional): when there are multiple elements with the same tag, use this to differentiate them. Defaults to 0.

**Raises:**

IndexError: Raised when a tag is not found. TlvError: Raised when value is not of type bytes.

**Returns:**

bytes: value

**get\_tlv**(*tag: int, index: int = 0*) → *TLV*

Similar to get\_value, but always returns a TLV object. Raises error when requested element is not a TLV object

**Args:**

tag (int): tag number index (int, optional): when there are multiple elements with the same tag, use this to differentiate them. Defaults to 0.

**Raises:**

IndexError: Raised when a tag is not found. TlvError: Raised when value is not of type TLV.

**Returns:**

TLV: value

**get\_value**(tag: int, index: int = 0) → bytes | *TLV*

Get the value of a tag. use index if there are multiple tags with the same name

**Args:**

tag (int): tag to find. index (int, optional): use to differentiate multiple values with the same tag. Defaults to 0.

**Raises:**

IndexError: Raised when a tag is not found.

**Returns:**

bytes | TLV: the value as bytes or TLV. Type depends on the structure of the TLV.

**to\_bytes**() → bytes

Converts tlv value (BER-TLV, ISO 7816-4) to bytestring. (length is derived from the value)

**Raises:**

TlvError: error during construction of the bytestring

**to\_data**() → list

returns the tlv as a list of tuples

**to\_print**() → str

returns a printable string.

**Returns:**

str: printable string with tags, values and lengths of this TLV

**exception** aliro\_actuator.access\_protocol.tlv.TlvError

Bases: *AccessProtocolError*

Raised when a TLV is malformed, has unexpected items or is missing items.

### aliro\_actuator.access\_protocol.user\_device module

**class** aliro\_actuator.access\_protocol.user\_device.MailboxSession

Bases: object

**class** Command(value)

Bases: Enum

An enumeration.

**SET** = 0

**WRITE** = 1

**add\_set**(data: bytes | None) → None

**add\_write**(data: bytes | None) → None

**execute\_commands**(mailbox: Mailbox) → None



**Raises:**

SessionError: Raised when the session is missing or in an invalid state. VersionError: Raised when the protocol version is not supported. NotImplementedError:

**async handle\_auth1**(*auth1\_command*: Command) → None

Parse a auth1 command and send the appropriate response.

**Args:**

auth1\_command (Command): The command to respond to.

**Raises:**

SessionError: Raised when the session is missing or in an invalid state. KeyLookupFailed: When the reader public key cannot be found. AccessProtocolError: Raised if the response has invalid data.

**async handle\_control\_flow**(*control\_flow\_command*: Command) → None

Parse an control flow command and send the appropriate response.

**Args:**

control\_flow\_command (Command): The command to respond to.

**Raises:**

SessionError: Raised when the session is missing or in an invalid state.

**async handle\_exchange**(*exchange\_command*: Command) → None

Parse an exchange command and send the appropriate response.

**Args:**

exchange\_command (Command): The command to respond to.

**Raises:**

SessionError: Raised when the session is missing or in an invalid state. AccessProtocolError: Raised if the response has invalid data.

**async handle\_load\_cert**(*load\_cert\_command*: Command) → None

Parse a load cert command and send the appropriate response.

**Args:**

load\_cert\_command (Command): The command to respond to.

**Raises:**

SessionError: Raised when the session is missing or in an invalid state. KeyLookupFailed: When the reader public key cannot be found.

**async handle\_select**(*select\_command*: Command) → bytes

Parse a select command and send the appropriate response.

**Args:**

select\_command (Command): The command to respond to.

**Raises:**

SessionError: Raised if no session is found. InvalidAIDError: Raised when the AID is invalid.

**async main\_loop**() → None

Starts a loop, where every command received is replied with an appropriate response. Should keep running, even when receiving invalid commands.

**Raises:**

SessionError: When starting a new session failed. NotImplementedError: When a command which is not implemented is received.

**async response\_auth0**(*credential\_epubk: bytes, cryptogram: bytes | None = None*) → None

Create and send an auth0 response.

**Args:**

*credential\_epubk* (bytes): Credential Ephemeral public key. *cryptogram* (bytes | None, optional): authentication cryptogram. Defaults to None.

**async response\_auth1**(*key\_slot: bytes | None, credential\_public\_key: bytes | None, expected\_response: Auth1Response, signature: bytes, encryption: EncryptionEngine, status: int = StatusBytes.SUCCESS, private\_mailbox\_data: bytes | None = None, signaling\_bitmap: bytes | None = None, credential\_signed\_timestamp: bytes | None = None, revocation\_signed\_timestamp: bytes | None = None*) → None

Create and send an auth1 response.

**Args:**

*key\_slot* (bytes | None): First 8 bytes of the keyIdentifier. *credential\_public\_key* (bytes | None): Credential long term public key. *expected\_response* (Auth1Response): expected response (keyslot or credential public key) signature (bytes): User device authentication signature. *encryption* (EncryptionEngine): Encryption engine to encrypt the response. *status* (int, optional): response status. Defaults to StatusBytes.SUCCESS. *private\_mailbox\_data* (bytes | None, optional): Defaults to None. *signaling\_bitmap* (bytes | None, optional): Defaults to None. *credential\_signed\_timestamp* (bytes | None, optional): Defaults to None. *revocation\_signed\_timestamp* (bytes | None, optional): Defaults to None.

**async response\_control\_flow**() → None

Create and send a control flow response.

**response\_envelope**() → None

**async response\_exchange**(*payload: bytes, encryption: EncryptionEngine*) → None

Create and send an exchange response.

**Args:**

*payload* (bytes): exchange response payload. *encryption* (EncryptionEngine): encryption engine to encrypt the payload.

**response\_get\_response**() → None

**async response\_load\_cert**() → None

Create and send a load cert response.

**async response\_select**(*aid: bytes, type: int, protocol\_versions: list[int], maximum\_command\_apdu: int | None = None, maximum\_response\_apdu: int | None = None, vendor\_specific\_tlv: TLV | None = None*) → None

Create and send a select response.

**Args:**

*aid* (bytes): AID type (int): Application type (table 10-4) *protocol\_versions* (list[int]): Expedited phase supported protocol versions. *maximum\_command\_apdu* (int | None, optional): Defaults to None. *maximum\_response\_apdu* (int | None, optional): Defaults to None. *vendor\_specific\_tlv* (TLV | None, optional): Defaults to None.

**async return\_exchange\_error\_and\_close\_channel**() → None

Return an exchange error and close the channel. Used when an exchange fails.

**Raises:**

SessionError: Raised if no session is found. AccessProtocolError: Raised if no encryption engine is found.

**async send\_initiate\_access\_protocol\_notification()** → None

Used by BLE, after a connection is established.

**async setup\_connection()** → None

Setup up the connection to the reader device.

**start\_new\_session()** → None

Start a new user session. Must be done before using handle commands. This sessions stores all information received from commands. Start a new session to delete all received info and start over.

**async transaction\_initiation()** → None

Initializes the hardware and sets up a connection to the reader.

**async transaction\_termination()** → None

terminates the connection to the reader.

**async wait\_for\_command**(*expected\_command*: *INS* | *list[INS]* | *None* = *None*, *encryption*: *EncryptionEngine* | *None* = *None*) → *Command*

Waits until a command is received, and parses the command.

#### Args:

*expected\_command* (*INS* | *list[INS]* | *None*, optional): *INS* or list of *INS* with expected commands. raises *UnexpectedCommandError* if another command is received. Defaults to *None*. *encryption* (*EncryptionEngine* | *None*, optional): Used for decrypting messages. Not required for every command. Defaults to *None*.

#### Raises:

*InvalidCLAError*: Raised when the received command has an invalid CLA. *InvalidParameterError*: Raised when the received command has an invalid Parameter (P1 or P2). *InvalidCommandError*: Raised when the received command is invalid. *VerificationError*: Raised when the verification of an AES decryption fails. *UnexpectedCommandError*: when the command is not in *expected\_command*.

#### Returns:

*Command*: the received command.

**class** aliro\_actuator.access\_protocol.user\_device.**UserMode**(*value*)

Bases: Enum

An enumeration.

**TEST** = 0

**USER** = 0

**class** aliro\_actuator.access\_protocol.user\_device.**UserSession**(*supported\_version*: *list[int]*,  
*vendor\_extension*: *bytes* | *None* = *None*)

Bases: object

Contains info from a single session (with one Reader Device)

**derive\_key\_persistent**(*transport\_protocol*: *TransportProtocol*) → bytes

**derive\_key\_volatile**(*transport\_protocol*: *TransportProtocol*) → None

**derive\_key\_volatile\_fast**(*transport\_protocol*: *TransportProtocol*, *k\_persistent*: bytes) → None

**generate\_ephemeral\_key**(*ephemeral\_key*: *KeyPair* | *None* = *None*) → None

```
get_credential_epubkey() → PublicKey
get_intermediate_reader_public_key() → PublicKey
get_reader_public_key() → PublicKey
get_transaction_type() → Transaction
property reader_group_identifier: bytes
property reader_group_sub_identifier: bytes
property reader_identifier: bytes
set_access_credential(access_credential: AccessCredential) → None
set_auth0_data(auth0_command: Command) → None
set_cert_and_verify(compressed_cert: bytes, public_key: PublicKey) → bool
set_shared_key() → None
state_valid(state: list[UserSessionState] | UserSessionState) → bool
update_state(state: UserSessionState) → None
```

```
class aliro_actuator.access_protocol.user_device.UserSessionState(value)
```

Bases: Enum

An enumeration.

```
AUTH0_FAST_DONE = 4
```

```
AUTH0_STD_DONE = 3
```

```
AUTH1_DONE = 5
```

```
EXCHANGE_DONE = 6
```

```
SELECT_DONE = 2
```

```
SELECT_STEP_UP_DONE = 7
```

```
SESSION_START = 1
```

```
class aliro_actuator.access_protocol.user_device.UserStorage
```

Bases: object

Cross-session storage for Expedited Fast cached data

```
add_kpersistent(kpersistent: bytes, reader_group_sub_id: bytes) → None
```

```
clear_kpersistent() → None
```

```
find_kpersistent(reader_group_sub_id: bytes) → bytes | None
```

```
remove_kpersistent(reader_group_sub_id: bytes) → None
```

## Module contents

### Access Protocol

Implements the access protocol part of the aliro protocol

```
class aliro_actuator.access_protocol.Device(transport_protocol: TransportProtocol, transport_override: TransportProtocolBase | None = None)
```

Bases: object

**property** support\_extended\_length\_apdu: bool

### aliro\_actuator.hw\_driver package

#### Subpackages

#### aliro\_actuator.hw\_driver.pn7160\_driver package

#### Submodules

#### aliro\_actuator.hw\_driver.pn7160\_driver.api module

```
class aliro_actuator.hw_driver.pn7160_driver.api.TECHNOLOGY_MASK(value)
```

Bases: IntEnum

An enumeration.

**DEFAULT** = -1

**MASK\_A** = 1

```
class aliro_actuator.hw_driver.pn7160_driver.api.ndef_info_t
```

Bases: Structure

**current\_ndef\_length**

Structure/Union member

**is\_def**

Structure/Union member

**is\_writable**

Structure/Union member

**max\_ndef\_length**

Structure/Union member

```
class aliro_actuator.hw_driver.pn7160_driver.api.nfcHostCardEmulationCallback_t
```

Bases: Structure

**onDataReceived**

Structure/Union member

**onHostCardEmulationActivated**

Structure/Union member

**onHostCardEmulationDeactivated**

Structure/Union member

**class** aliro\_actuator.hw\_driver.pn7160\_driver.api.nfcTagCallback\_t

Bases: Structure

**onTagArrival**

Structure/Union member

**onTagDeparture**

Structure/Union member

**class** aliro\_actuator.hw\_driver.pn7160\_driver.api.nfc\_tag\_info\_t

Bases: Structure

**handle**

Structure/Union member

**protocol**

Structure/Union member

**technology**

Structure/Union member

**uid**

Structure/Union member

**uid\_length**

Structure/Union member

## **aliro\_actuator.hw\_driver.pn7160\_driver.errors module**

**exception** aliro\_actuator.hw\_driver.pn7160\_driver.errors.DriverNotInitializedError(*error\_message: str | None = None*)

Bases: *PN7160DriverError*

**exception** aliro\_actuator.hw\_driver.pn7160\_driver.errors.NCIErrors(*error\_code: int, error\_message: str | None = None*)

Bases: *PN7160DriverError*

**exception** aliro\_actuator.hw\_driver.pn7160\_driver.errors.NCINotFoundError

Bases: *PN7160DriverError*

**exception** aliro\_actuator.hw\_driver.pn7160\_driver.errors.NoDataReceivedError

Bases: *PN7160DriverError*

**exception** aliro\_actuator.hw\_driver.pn7160\_driver.errors.NoReaderError(*error\_message: str | None = None*)

Bases: *PN7160DriverError*

**exception** aliro\_actuator.hw\_driver.pn7160\_driver.errors.NoTagError(*error\_message: str | None = None*)

Bases: *PN7160DriverError*

**exception** aliro\_actuator.hw\_driver.pn7160\_driver.errors.PN7160DriverError

Bases: Exception

## Module contents

**class** aliro\_actuator.hw\_driver.pn7160\_driver.Driver(nci\_location: str | None = None)

Bases: object

**disconnect**() → None

**initialize**(mode: Mode) → None

**receive\_message**() → bytes

**send\_message**(message: bytes) → None

**wait\_for\_reader**() → None

**wait\_for\_tag**() → None

## Module contents

aliro\_actuator.transport\_protocol package

## Submodules

aliro\_actuator.transport\_protocol.ble\_uwb module

**class** aliro\_actuator.transport\_protocol.ble\_uwb.BLEUWB(port: str | None = None)

Bases: TransportProtocolBase

**async disconnect**() → None

**async get\_message**(expected\_type: MessageType = MessageType.ANY) → bytes

**async initialization**(mode: Mode, reader\_group\_identifier: bytes = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',  
reader\_group\_sub\_identifier: bytes = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',  
group\_resolving\_key: bytes = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',  
reader\_group\_identifier\_list: list = [], spsm: bytes = b'\x00\x80') → None

**async send\_message**(command: bytes, type: MessageType) → None

**async wait\_for\_connection**() → None

**aliro\_actuator.transport\_protocol.errors module****exception** aliro\_actuator.transport\_protocol.errors.InvalidModeErrorBases: *TransportProtocolError***exception** aliro\_actuator.transport\_protocol.errors.NoDataReceivedErrorBases: *TransportProtocolError***exception** aliro\_actuator.transport\_protocol.errors.NoDeviceConnectedErrorBases: *TransportProtocolError***exception** aliro\_actuator.transport\_protocol.errors.TransportProtocolError

Bases: Exception

**exception** aliro\_actuator.transport\_protocol.errors.UnexpectedMessageTypeErrorBases: *TransportProtocolError***exception** aliro\_actuator.transport\_protocol.errors.UnknownVersionRequestedErrorBases: *TransportProtocolError***aliro\_actuator.transport\_protocol.nfc module****class** aliro\_actuator.transport\_protocol.nfc.NFC(*port: str | None = None*)Bases: *TransportProtocolBase***async disconnect()** → None**async get\_message**(*expected\_type: MessageType = MessageType.ANY*) → bytes**async initialization**(*mode: Mode, reader\_group\_identifier: bytes = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00', reader\_group\_sub\_identifier: bytes = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00', group\_resolving\_key: bytes = b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00', reader\_group\_identifier\_list: list = [], spsm: bytes = b'\x00\x80'*) → None**async send\_message**(*command: bytes, type: MessageType*) → None**async wait\_for\_connection()** → None**aliro\_actuator.transport\_protocol.socket module****class** aliro\_actuator.transport\_protocol.socket.SocketBases: *TransportProtocolBase*

Uses sockets to communicate. Mainly used for testing purposes.

**disconnect()** → None**async get\_message**(*expected\_type: MessageType = MessageType.ANY*) → bytes

```

async initialization(mode: Mode, reader_group_identifier: bytes =
    b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',
    reader_group_sub_identifier: bytes =
    b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',
    group_resolving_key: bytes =
    b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',
    reader_group_identifier_list: list = [], spsm: bytes = b'\x00\x80') → None

async send_message(command: bytes, type: MessageType) → None

async wait_for_connection() → None

```

## Module contents

```
class aliro_actuator.transport_protocol.MessageType(value)
```

Bases: Enum

An enumeration.

**ANY** = 0

**INITIATE\_ACCESS\_PROTOCOL** = 3

**REQUEST** = 1

**RESPONSE** = 2

```
class aliro_actuator.transport_protocol.Mode(value)
```

Bases: Enum

An enumeration.

**READER** = 1

**USER\_DEVICE** = 2

```
class aliro_actuator.transport_protocol.TransportProtocolBase
```

Bases: ABC

**abstract async disconnect**() → None

**abstract async get\_message**(expected\_type: [MessageType](#) = [MessageType](#).ANY) → bytes

```

abstract async initialization(mode: Mode, reader_group_identifier: bytes =
    b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',
    reader_group_sub_identifier: bytes =
    b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',
    group_resolving_key: bytes =
    b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00',
    reader_group_identifier_list: list = [], spsm: bytes = b'\x00\x80') →
    None

```

**abstract async send\_message**(command: bytes, type: [MessageType](#)) → None

**abstract async wait\_for\_connection**() → None

## aliro\_actuator.trust\_framework package

### Submodules

#### aliro\_actuator.trust\_framework.access\_credential module

```
class aliro_actuator.trust_framework.access_credential.AccessCredential(user_device_key_pair:
                                                                    KeyPair,
                                                                    reader_id_key_list:
                                                                    list[tuple[bytes,
                                                                    PublicKey]])
```

Bases: object

`_summary_`

#### Args:

`user_device_key_pair` (*KeyPair*): keypair used by the User Device when this *AccessCredential* has a matching reader group identifier  
`reader_id_key_list` (*list[tuple[bytes, PublicKey]]*): a list with tuples containing reader\_group\_identifier and reader\_public\_key pairs

**get\_all\_reader\_id()** → *list[bytes]*

**get\_credential\_public\_key()** → *PublicKey*

**get\_key\_slot()** → *bytes*

**get\_reader\_public\_key(identifier: bytes)** → *PublicKey*

**has\_identifier(group\_identifier: bytes)** → *bool*

Checks if this *AccessCredential* has the given reader group identifier, and the reader public key can be used for this reader device.

#### Args:

`group_identifier` (*bytes*): reader group identifier to check.

#### Returns:

*bool*: True if this *access\_credential* has the given reader group identifier.

**sign(data: bytes)** → *bytes*

#### aliro\_actuator.trust\_framework.certificate module

```
class aliro_actuator.trust_framework.certificate.Certificate(key_info_subject_public_key: bytes,
                                                            signature: bytes, serial_number:
                                                            bytes = b'\x01', issuer: bytes =
                                                            b'issuer', validity_not_before: bytes
                                                            = b'200101000000Z',
                                                            validity_not_after: bytes =
                                                            b'490101000000Z', subject: bytes =
                                                            b'subject')
```

Bases: object

**classmethod decode(certificate: bytes)** → *Certificate*

**classmethod decode\_compressed(compressed\_certificate: bytes)** → *Certificate*

```

default_issuer = b'issuer'
default_serial_number = b'\x01'
default_subject = b'subject'
default_validity_not_after = b'490101000000Z'
default_validity_not_before = b'200101000000Z'

encode(issuer_public_key: PublicKey) → bytes
encode_compressed() → bytes

static generate(key_info_subject_public_key: bytes, issuer_keypair: KeyPair, serial_number: bytes =
    b'\x01', issuer: bytes = b'issuer', validity_not_before: bytes = b'200101000000Z',
    validity_not_after: bytes = b'490101000000Z', subject: bytes = b'subject') → bytes
    Generate a new X509 DER certificate signed by the provided issuer keypair

get_public_key() → PublicKey

verify(key: PublicKey) → bool

```

### aliro\_actuator.trust\_framework.errors module

```

exception aliro_actuator.trust_framework.errors.CertificateDecodingError(certificate: bytes,
                                                                           message: str | None = None)

```

Bases: *TrustFrameworkError*

Passed when a certificate cannot be decoded.

```

exception aliro_actuator.trust_framework.errors.InvalidIdentifierError(identifier: bytes,
                                                                           message: str | None = None)

```

Bases: *TrustFrameworkError*

Passed when a identifier is invalid.

```

exception aliro_actuator.trust_framework.errors.InvalidKeyError(key: bytes, message: str | None = None)

```

Bases: *TrustFrameworkError*

Passed when a key is invalid.

```

exception aliro_actuator.trust_framework.errors.InvalidKeyFormatError

```

Bases: *TrustFrameworkError*

Passed when a key has the wrong format.

```

exception aliro_actuator.trust_framework.errors.KeyLookupFailed

```

Bases: *TrustFrameworkError*

```

exception aliro_actuator.trust_framework.errors.MissingPublicKeyError

```

Bases: *TrustFrameworkError*

Passed when initializing private key with 32 bytes data, and no public key.

**exception** `aliro_actuator.trust_framework.errors.TrustFrameworkError`

Bases: `Exception`

Parent class of all errors related to the trust framework.

### **aliro\_actuator.trust\_framework.key module**

**class** `aliro_actuator.trust_framework.key.Key`

Bases: `object`

Base class for all keys.

**class** `aliro_actuator.trust_framework.key.KeyPair`(*private\_key*: *str* | *bytes* | `PrivateKey` | *None* = *None*,  
*public\_key*: *bytes* | *str* | `PublicKey` | *None* = *None*)

Bases: `object`

Combination of a private and public key.

**get\_private\_key**() → `PrivateKey`

Returns the private key.

**get\_public\_key**() → `PublicKey`

Returns the public key.

**get\_public\_key\_as\_bytes**() → *bytes*

Returns the public key as raw bytes. Prepend with 0x04, total length of 65.

**sign**(*data*: *bytes*) → *bytes*

Sign the data. The signature is returned.

(figure 8-5 of Aliro spec)

**verify**(*data*: *bytes*, *signature*: *bytes*) → *bool*

Verifies the data. Returns True when the verification succeeds.

(figure 8-6 of Aliro spec)

**class** `aliro_actuator.trust_framework.key.PrivateKey`(*key*: *bytes* | *str* | *None* = *None*, *public\_key*: *bytes*  
| *None* = *None*)

Bases: `Key`

Private Key

**as\_bytes**(*short*: *bool* = *True*) → *bytes*

Returns the key as raw bytes.

**as\_pem**() → *str*

Returns the key as PEM.

**compute\_shared\_key**(*public\_key*: `PublicKey`, *shared\_info*: *bytes*) → *bytes*

Compute a shared key using Diffie-Hellman

(figure 8-7 of the Aliro spec)

**generate\_public\_key**() → `PublicKey`

Generate a public key from this private key.

**sign**(data: bytes) → bytes

Sign the data. The signature is returned.

(figure 8-5 of Aliro spec)

**class** aliro\_actuator.trust\_framework.key.PublicKey(key: bytes | EllipticCurvePublicKey | str)

Bases: [Key](#)

Public Key

**as\_bytes**() → bytes

Returns the key as raw bytes. Prepended with 0x04, total length of 65.

**as\_pem**() → str

Returns the key as PEM.

**get\_x**() → int

**get\_y**() → int

**verify**(data: bytes, signature: bytes) → bool

Verifies the data. Returns True when the verification succeeds.

(figure 8-6 of Aliro spec)

aliro\_actuator.trust\_framework.key.derive\_key(input\_key: bytes, info: bytes, length: int, salt: bytes) → bytes

Derive key

(figure 8-8 of the Aliro spec)

### aliro\_actuator.trust\_framework.reader\_identifier module

**class** aliro\_actuator.trust\_framework.reader\_identifier.ReaderIdentifier(identifier: bytes)

Bases: object

**as\_bytes**() → bytes

**get\_group**() → bytes

**get\_group\_sub**() → bytes

## Module contents

### 3.1.1.2 Submodules

#### aliro\_actuator.log module

**class** aliro\_actuator.log.InterceptHandler(log\_source: str)

Bases: Handler

This class is used to intercept the standard python logging. To enable it, you must call.

This is based on boilerplate code from the loguru project

`logging.basicConfig(handlers=[InterceptHandler()], level=0)`

**emit**(*record: LogRecord*) → None

This will be called by the standard logging module on each log call

**Args:**

record (logging.LogRecord): The log record intercepted from the standard logging module

**loglevel\_mapping** = {0: 'NOTSET', 10: 'DEBUG', 20: 'INFO', 30: 'WARNING', 40: 'ERROR', 50: 'CRITICAL'}

**exception** aliro\_actuator.log.LoggingConfigError

Bases: Exception

Raised when config parameters are missing or don't align

**aliro\_actuator.log.configure\_logging**() → None

### 3.1.1.3 Module contents

**class** aliro\_actuator.Global

Bases: object

**logger** = <loguru.logger handlers=[(id=1, level=10, sink=<stdout>), (id=2, level=10, sink='/app/src/./logs/{time:YYYY-MM-DD--HH-mm-ss}.log')]>

**classmethod** **set\_logger**(*logger*) → None

## INDICES AND TABLES

- `genindex`
- `modindex`
- `search`



## PYTHON MODULE INDEX

### a

- `aliro_actuator`, 44
- `aliro_actuator.access_document`, 8
- `aliro_actuator.access_document.access_credential`, 7
- `aliro_actuator.access_document.errors`, 7
- `aliro_actuator.access_document.revocation_document`, 8
- `aliro_actuator.access_protocol`, 35
- `aliro_actuator.access_protocol.apdu`, 8
- `aliro_actuator.access_protocol.authentication`, 13
- `aliro_actuator.access_protocol.defines`, 14
- `aliro_actuator.access_protocol.encryption`, 17
- `aliro_actuator.access_protocol.errors`, 19
- `aliro_actuator.access_protocol.mailbox`, 21
- `aliro_actuator.access_protocol.reader`, 22
- `aliro_actuator.access_protocol.tlv`, 27
- `aliro_actuator.access_protocol.user_device`, 29
- `aliro_actuator.hw_driver`, 37
- `aliro_actuator.hw_driver.pn7160_driver`, 37
- `aliro_actuator.hw_driver.pn7160_driver.api`, 35
- `aliro_actuator.hw_driver.pn7160_driver.errors`, 36
- `aliro_actuator.log`, 43
- `aliro_actuator.transport_protocol`, 39
- `aliro_actuator.transport_protocol.ble_uwb`, 37
- `aliro_actuator.transport_protocol.errors`, 38
- `aliro_actuator.transport_protocol.nfc`, 38
- `aliro_actuator.transport_protocol.socket`, 38
- `aliro_actuator.trust_framework`, 43
- `aliro_actuator.trust_framework.access_credential`, 40
- `aliro_actuator.trust_framework.certificate`, 40
- `aliro_actuator.trust_framework.errors`, 41
- `aliro_actuator.trust_framework.key`, 42
- `aliro_actuator.trust_framework.reader_identifier`, 43



## INDEX

### A

|                                                                 |                                                                                         |                                                                             |
|-----------------------------------------------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>AccessCredential</code>                                   | (class in <code>aliro_actuator.trust_framework.access_credential</code> ),<br>40        | <code>aliro_actuator.access_protocol.errors</code><br>module, 19            |
| <code>AccessDocument</code>                                     | (class in <code>aliro_actuator.access_document.access_credential</code> ),<br>7         | <code>aliro_actuator.access_protocol.mailbox</code><br>module, 21           |
| <code>AccessProtocolError</code>                                | 19                                                                                      | <code>aliro_actuator.access_protocol.reader</code><br>module, 22            |
| <code>add_kpersistent()</code>                                  | ( <code>aliro_actuator.access_protocol.reader.ReaderStorage</code><br>method), 27       | <code>aliro_actuator.access_protocol.tlv</code><br>module, 27               |
| <code>add_kpersistent()</code>                                  | ( <code>aliro_actuator.access_protocol.user_device.UserStorage</code><br>method), 34    | <code>aliro_actuator.access_protocol.user_device</code><br>module, 29       |
| <code>add_set()</code>                                          | ( <code>aliro_actuator.access_protocol.user_device.MailboxSession</code><br>method), 29 | <code>aliro_actuator.hw_driver</code><br>module, 37                         |
| <code>add_value()</code>                                        | ( <code>aliro_actuator.access_protocol.tlv.TLV</code><br>method), 27                    | <code>aliro_actuator.hw_driver.pn7160_driver</code><br>module, 37           |
| <code>add_write()</code>                                        | ( <code>aliro_actuator.access_protocol.user_device.MailboxSession</code><br>method), 29 | <code>aliro_actuator.hw_driver.pn7160_driver.api</code><br>module, 35       |
| <code>AID_LEN</code>                                            | ( <code>aliro_actuator.access_protocol.defines.Select</code><br>attribute), 16          | <code>aliro_actuator.hw_driver.pn7160_driver.errors</code><br>module, 36    |
| <code>AID_TAG</code>                                            | ( <code>aliro_actuator.access_protocol.defines.Select</code><br>attribute), 16          | <code>aliro_actuator.log</code><br>module, 43                               |
| <code>aliro_actuator</code>                                     | module, 44                                                                              | <code>aliro_actuator.transport_protocol</code><br>module, 39                |
| <code>aliro_actuator.access_document</code>                     | module, 8                                                                               | <code>aliro_actuator.transport_protocol.ble_uwb</code><br>module, 37        |
| <code>aliro_actuator.access_document.access_credential</code>   | module, 7                                                                               | <code>aliro_actuator.transport_protocol.errors</code><br>module, 38         |
| <code>aliro_actuator.access_document.errors</code>              | module, 7                                                                               | <code>aliro_actuator.transport_protocol.nfc</code><br>module, 38            |
| <code>aliro_actuator.access_document.revocation_document</code> | module, 8                                                                               | <code>aliro_actuator.transport_protocol.socket</code><br>module, 38         |
| <code>aliro_actuator.access_protocol</code>                     | module, 35                                                                              | <code>aliro_actuator.trust_framework</code><br>module, 43                   |
| <code>aliro_actuator.access_protocol.apdu</code>                | module, 8                                                                               | <code>aliro_actuator.trust_framework.access_credential</code><br>module, 40 |
| <code>aliro_actuator.access_protocol.authentication</code>      | module, 13                                                                              | <code>aliro_actuator.trust_framework.certificate</code><br>module, 40       |
| <code>aliro_actuator.access_protocol.defines</code>             | module, 14                                                                              | <code>aliro_actuator.trust_framework.errors</code><br>module, 41            |
| <code>aliro_actuator.access_protocol.encryption</code>          | module, 17                                                                              | <code>aliro_actuator.trust_framework.key</code><br>module, 42               |
|                                                                 |                                                                                         | <code>aliro_actuator.trust_framework.reader_identifier</code><br>module, 43 |

ANY (*aliro\_actuator.transport\_protocol.MessageType* attribute), 39

APDU (*class* in *aliro\_actuator.access\_protocol.apdu*), 8

as\_bytes() (*aliro\_actuator.trust\_framework.key.PrivateKey* method), 42

as\_bytes() (*aliro\_actuator.trust\_framework.key.PublicKey* method), 43

as\_bytes() (*aliro\_actuator.trust\_framework.reader\_identifier.ReaderIdentifier* method), 43

as\_pem() (*aliro\_actuator.trust\_framework.key.PrivateKey* method), 42

as\_pem() (*aliro\_actuator.trust\_framework.key.PublicKey* method), 43

AUTH0 (*aliro\_actuator.access\_protocol.apdu.INS* attribute), 10

Auth0 (*class* in *aliro\_actuator.access\_protocol.defines*), 14

AUTH0\_FAST\_DONE (*aliro\_actuator.access\_protocol.user\_device.UserDevice* attribute), 34

AUTH0\_STD\_DONE (*aliro\_actuator.access\_protocol.user\_device.UserDevice* attribute), 34

AUTH1 (*aliro\_actuator.access\_protocol.apdu.INS* attribute), 10

Auth1 (*class* in *aliro\_actuator.access\_protocol.defines*), 14

AUTH1\_DONE (*aliro\_actuator.access\_protocol.user\_device.UserDevice* attribute), 34

Auth1Response (*class* in *aliro\_actuator.access\_protocol.apdu*), 9

**B**

BLE\_UWB (*aliro\_actuator.access\_protocol.defines.TransportProtocol* attribute), 16

BLEUWB (*class* in *aliro\_actuator.transport\_protocol.ble\_uwb*), 37

**C**

can\_retrieve\_access\_credential() (*aliro\_actuator.access\_protocol.reader.ReaderSession* method), 26

can\_retrieve\_revocation\_document() (*aliro\_actuator.access\_protocol.reader.ReaderSession* method), 26

Certificate (*class* in *aliro\_actuator.trust\_framework.certificate*), 40

CERTIFICATE\_TAG (*aliro\_actuator.access\_protocol.defines.Auth1* attribute), 14

CertificateDecodingError, 41

check\_boundaries() (*aliro\_actuator.access\_protocol.mailbox.Mailbox* method), 21

check\_counters\_valid() (*aliro\_actuator.access\_protocol.encryption.EncryptionEngine* method), 17

check\_reader\_authentication\_data() (*aliro\_actuator.access\_protocol.user\_device.UserDevice* method), 30

check\_user\_device\_authentication() (*aliro\_actuator.access\_protocol.reader.ReaderSession* method), 26

clear\_kpersistent() (*aliro\_actuator.access\_protocol.reader.ReaderStorage* method), 27

clear\_kpersistent() (*aliro\_actuator.access\_protocol.user\_device.UserStorage* method), 34

Command (*class* in *aliro\_actuator.access\_protocol.apdu*), 9

command\_auth0() (*aliro\_actuator.access\_protocol.reader.Reader* method), 23

command\_auth1() (*aliro\_actuator.access\_protocol.reader.Reader* method), 23

command\_control\_flow() (*aliro\_actuator.access\_protocol.reader.Reader* method), 23

command\_envelope() (*aliro\_actuator.access\_protocol.reader.Reader* method), 23

command\_exchange() (*aliro\_actuator.access\_protocol.reader.Reader* method), 23

command\_get\_response() (*aliro\_actuator.access\_protocol.reader.Reader* method), 24

COMMAND\_LEN (*aliro\_actuator.access\_protocol.defines.Auth0* attribute), 14

COMMAND\_LEN (*aliro\_actuator.access\_protocol.defines.Auth1* attribute), 14

command\_load\_cert() (*aliro\_actuator.access\_protocol.reader.Reader* method), 24

COMMAND\_NOT\_ALLOWED (*aliro\_actuator.access\_protocol.apdu.StatusBytes* attribute), 12

COMMAND\_NOT\_COMPLIANT (*aliro\_actuator.access\_protocol.apdu.StatusBytes* attribute), 12

command\_select() (*aliro\_actuator.access\_protocol.reader.Reader* method), 24

COMMAND\_TAG (*aliro\_actuator.access\_protocol.defines.Auth0* attribute), 14

COMMAND\_TAG (*aliro\_actuator.access\_protocol.defines.Auth1* attribute), 14

compute\_cryptogram() (*in* module *aliro\_actuator.access\_protocol.encryption*), 18

compute\_shared\_key() (*aliro\_actuator.trust\_framework.key.PrivateKey* method), 42

CONDITIONS\_OF\_USE\_NOT\_SATISFIED (*aliro\_actuator.access\_protocol.apdu.StatusBytes* attribute), 12

attribute), 12  
 configure\_logging() (in module *aliro\_actuator.log*), 44  
 CONTROL\_FLOW (*aliro\_actuator.access\_protocol.apdu.INS* attribute), 10  
 ControlFlow (class in *aliro\_actuator.access\_protocol.defines*), 15  
 create\_auth0\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 8  
 create\_auth0\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 8  
 create\_auth1\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 8  
 create\_auth1\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 8  
 create\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 8  
 create\_control\_flow\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 8  
 create\_control\_flow\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_envelope\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_envelope\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_error\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_exchange\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_exchange\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_from\_bytestring() (*aliro\_actuator.access\_protocol.apdu.Command* class method), 9  
 create\_from\_bytestring() (*aliro\_actuator.access\_protocol.apdu.Response* class method), 11  
 create\_from\_parameters() (*aliro\_actuator.access\_protocol.apdu.Command* class method), 9  
 create\_from\_parameters() (*aliro\_actuator.access\_protocol.apdu.Response* class method), 11  
 create\_get\_response\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_get\_response\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_load\_cert\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_load\_cert\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_proprietary\_information() (in module *aliro\_actuator.access\_protocol.encryption*), 18  
 create\_reader\_authentication() (in module *aliro\_actuator.access\_protocol.authentication*), 13  
 create\_salt() (in module *aliro\_actuator.access\_protocol.encryption*), 18  
 create\_select\_command() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_select\_response() (*aliro\_actuator.access\_protocol.apdu.APDU* method), 9  
 create\_user\_device\_authentication() (in module *aliro\_actuator.access\_protocol.authentication*), 13  
 CreateCommandError, 19  
 CreateResponseError, 19  
 CREDENTIAL\_EPUBK\_LEN (*aliro\_actuator.access\_protocol.defines.Auth0* attribute), 14  
 CREDENTIAL\_EPUBK\_TAG (*aliro\_actuator.access\_protocol.defines.Auth0* attribute), 14  
 CREDENTIAL\_EPUBK\_TAG (*aliro\_actuator.access\_protocol.defines.ReaderAuth* attribute), 15  
 CREDENTIAL\_EPUBK\_TAG (*aliro\_actuator.access\_protocol.defines.UserDeviceAuth* attribute), 16  
 CREDENTIAL\_PUBK\_LEN (*aliro\_actuator.access\_protocol.defines.Auth1* attribute), 14  
 CREDENTIAL\_PUBK\_TAG (*aliro\_actuator.access\_protocol.defines.Auth1* attribute), 14  
 CREDENTIAL\_PUBLIC\_KEY (*aliro\_actuator.access\_protocol.apdu.Auth1Response* attribute), 9  
 CREDENTIAL\_TIMESTAMP\_LEN (*aliro\_actuator.access\_protocol.defines.Auth1* attribute), 14

**CREDENTIAL\_TIMESTAMP\_TAG** (*aliro\_actuator.access\_protocol.defines.Auth1 attribute*), 14  
**CRYPTOGRAM\_LEN** (*aliro\_actuator.access\_protocol.defines.Auth1 attribute*), 14  
**CRYPTOGRAM\_TAG** (*aliro\_actuator.access\_protocol.defines.Auth0 attribute*), 14  
**CryptogramNotFound**, 19  
**current\_ndef\_length** (*aliro\_actuator.hw\_driver.pn7160\_driver.api.ndef.ndef attribute*), 35  
**D**  
**data\_is\_set()** (*aliro\_actuator.access\_protocol.mailbox.Mailbox method*), 21  
**DATA\_TAG** (*aliro\_actuator.access\_protocol.mailbox.Mailbox attribute*), 21  
**DataElement** (class in *aliro\_actuator.access\_document.access\_credentials*), 7  
**decode()** (*aliro\_actuator.trust\_framework.certificate.Certificate class method*), 40  
**decode\_compressed()** (*aliro\_actuator.trust\_framework.certificate.Certificate class method*), 40  
**decrypt()** (*aliro\_actuator.access\_protocol.encryption.EncryptionEngine method*), 17  
**decrypt\_cryptogram()** (*aliro\_actuator.access\_protocol.reader.Reader method*), 24  
**decrypt\_cryptogram()** (in module *aliro\_actuator.access\_protocol.encryption*), 18  
**decrypt\_payload()** (*aliro\_actuator.access\_protocol.reader.ReaderSession method*), 26  
**DEFAULT** (*aliro\_actuator.hw\_driver.pn7160\_driver.api.TECHNOLOGY\_MASK attribute*), 35  
**default\_issuer** (*aliro\_actuator.trust\_framework.certificate.Certificate attribute*), 40  
**default\_serial\_number** (*aliro\_actuator.trust\_framework.certificate.Certificate attribute*), 41  
**default\_subject** (*aliro\_actuator.trust\_framework.certificate.Certificate attribute*), 41  
**default\_validity\_not\_after** (*aliro\_actuator.trust\_framework.certificate.Certificate attribute*), 41  
**default\_validity\_not\_before** (*aliro\_actuator.trust\_framework.certificate.Certificate attribute*), 41  
**derive\_key()** (in module *aliro\_actuator.trust\_framework.key*), 43  
**derive\_key\_persistent()** (*aliro\_actuator.access\_protocol.reader.ReaderSession method*), 26  
**derive\_key\_persistent()** (*aliro\_actuator.access\_protocol.user\_device.UserSession method*), 33  
**derive\_key\_volatile()** (*aliro\_actuator.access\_protocol.reader.ReaderSession method*), 26  
**derive\_key\_volatile()** (*aliro\_actuator.access\_protocol.user\_device.UserSession method*), 33  
**derive\_key\_volatile\_fast()** (*aliro\_actuator.access\_protocol.reader.ReaderSession method*), 26  
**derive\_key\_volatile\_fast()** (*aliro\_actuator.access\_protocol.user\_device.UserSession method*), 33  
**Device** (class in *aliro\_actuator.access\_protocol*), 35  
**DeviceType** (class in *aliro\_actuator.access\_protocol.encryption*), 17  
**disconnect()** (*aliro\_actuator.hw\_driver.pn7160\_driver.Driver method*), 37  
**disconnect()** (*aliro\_actuator.transport\_protocol.ble\_uwb.BLEUWB method*), 37  
**disconnect()** (*aliro\_actuator.transport\_protocol.nfc.NFC method*), 38  
**disconnect()** (*aliro\_actuator.transport\_protocol.socket.Socket method*), 38  
**disconnect()** (*aliro\_actuator.transport\_protocol.TransportProtocolBase method*), 39  
**Document** (class in *aliro\_actuator.access\_document*), 8  
**DocumentError**, 7  
**DOMAIN\_SPECIFIC\_TAG** (*aliro\_actuator.access\_protocol.defines.ControlFlow attribute*), 15  
**Driver** (class in *aliro\_actuator.hw\_driver.pn7160\_driver*), 37  
**DriverNotInitializedError**, 36  
**E**  
**ElementSizeError**, 7  
**emit()** (*aliro\_actuator.log.InterceptHandler method*), 43  
**encode()** (*aliro\_actuator.trust\_framework.certificate.Certificate method*), 41  
**encode\_compressed()** (*aliro\_actuator.trust\_framework.certificate.Certificate method*), 41  
**encrypt()** (*aliro\_actuator.access\_protocol.encryption.EncryptionEngine method*), 17  
**encrypt\_payload()** (*aliro\_actuator.access\_protocol.reader.ReaderSession method*), 26  
**EncryptionEngine** (class in *aliro\_actuator.access\_protocol.encryption*), 17  
**end\_session()** (*aliro\_actuator.access\_protocol.reader.Reader method*), 24

end\_session() (aliro\_actuator.access\_protocol.user\_device.UserSession attribute), 30

ENVELOPE (aliro\_actuator.access\_protocol.apdu.INS attribute), 11

ETPV\_LEN (aliro\_actuator.access\_protocol.defines.Auth0 attribute), 14

ETPV\_TAG (aliro\_actuator.access\_protocol.defines.Auth0 attribute), 14

ETSPV\_TAG (aliro\_actuator.access\_protocol.defines.Select attribute), 16

EXCHANGE (aliro\_actuator.access\_protocol.apdu.INS attribute), 11

Exchange (class in aliro\_actuator.access\_protocol.defines), 15

EXCHANGE\_DONE (aliro\_actuator.access\_protocol.user\_device.UserSession attribute), 34

execute\_commands() (aliro\_actuator.access\_protocol.user\_device.MailboxSession method), 29

expedited\_transaction\_fast() (aliro\_actuator.access\_protocol.reader.Reader method), 24

expedited\_transaction\_standard() (aliro\_actuator.access\_protocol.reader.Reader method), 24

EXTENDED\_INFO\_LEN (aliro\_actuator.access\_protocol.defines.Select attribute), 16

EXTENDED\_INFO\_TAG (aliro\_actuator.access\_protocol.defines.Select attribute), 16

**F**

failure\_process() (aliro\_actuator.access\_protocol.reader.Reader method), 24

failure\_process() (aliro\_actuator.access\_protocol.user\_device.Device method), 30

FAST (aliro\_actuator.access\_protocol.apdu.Transaction attribute), 13

FCI\_TAG (aliro\_actuator.access\_protocol.defines.Select attribute), 16

FILE\_OR\_APP\_NOT\_FOUND (aliro\_actuator.access\_protocol.apdu.StatusBytes attribute), 12

find\_kpersistent() (aliro\_actuator.access\_protocol.user\_device.UserStorage method), 34

FINISHED\_WITH\_FAILURE (aliro\_actuator.access\_protocol.apdu.SI attribute), 12

FINISHED\_WITH\_SUCCESS (aliro\_actuator.access\_protocol.apdu.SI attribute), 12

FORCE\_USER\_AUTHENTICATION (aliro\_actuator.access\_protocol.apdu.TransactionCode attribute), 13

from\_bytes() (aliro\_actuator.access\_protocol.tlv.TLV static method), 27

FUNCTION\_NOT\_SUPPORTED (aliro\_actuator.access\_protocol.apdu.StatusBytes attribute), 12

FUNCTIONS\_IN\_CLA\_NOT\_SUPPORTED (aliro\_actuator.access\_protocol.apdu.StatusBytes attribute), 12

**G**

generate() (aliro\_actuator.trust\_framework.certificate.Certificate static method), 41

generate\_ephemeral\_key() (aliro\_actuator.access\_protocol.reader.ReaderSession method), 26

generate\_ephemeral\_key() (aliro\_actuator.access\_protocol.user\_device.UserSession method), 33

generate\_public\_key() (aliro\_actuator.trust\_framework.key.PrivateKey method), 42

GENERIC\_ERROR (aliro\_actuator.access\_protocol.apdu.StatusBytes attribute), 12

get\_all\_bytes\_of\_tag() (aliro\_actuator.access\_protocol.tlv.TLV method), 28

get\_all\_of\_tag() (aliro\_actuator.access\_protocol.tlv.TLV method), 28

get\_all\_reader\_id() (aliro\_actuator.trust\_framework.access\_credential.AccessCredential method), 40

get\_all\_tlv\_of\_tag() (aliro\_actuator.access\_protocol.tlv.TLV method), 28

get\_bytes\_of() (aliro\_actuator.access\_protocol.tlv.TLV method), 28

get\_credential\_ephemeral\_key() (aliro\_actuator.access\_protocol.reader.ReaderSession method), 26

get\_credential\_epubkey() (aliro\_actuator.access\_protocol.user\_device.UserSession method), 33

get\_credential\_public\_key() (aliro\_actuator.trust\_framework.access\_credential.AccessCredential method), 40

get\_data() (aliro\_actuator.access\_protocol.mailbox.Mailbox method), 21

get\_group() (aliro\_actuator.trust\_framework.reader\_identifier.ReaderIdentifier method), 43

get\_group\_sub() (aliro\_actuator.trust\_framework.reader\_identifier.ReaderIdentifier method), 43

get\_intermediate\_reader\_public\_key() (aliro\_actuator.access\_protocol.user\_device.UserSession method), 34

get\_key\_slot() (aliro\_actuator.trust\_framework.access\_credential.AccessCredential method), 40

get\_kpersistent\_list()

(aliro\_actuator.access\_protocol.reader.ReaderStorage, method), 27

get\_message() (aliro\_actuator.transport\_protocol.ble\_uwb.BLEUWB, method), 37

get\_message() (aliro\_actuator.transport\_protocol.nfc.NFC, method), 38

get\_message() (aliro\_actuator.transport\_protocol.socket.Socket, method), 38

get\_message() (aliro\_actuator.transport\_protocol.TransportProtocolBase, method), 39

get\_private\_key() (aliro\_actuator.trust\_framework.key.KeyPair, method), 42

get\_public\_key() (aliro\_actuator.trust\_framework.certificate.Certificate, method), 41

get\_public\_key() (aliro\_actuator.trust\_framework.key.KeyPair, method), 42

get\_public\_key\_as\_bytes() (aliro\_actuator.trust\_framework.key.KeyPair, method), 42

get\_raw() (aliro\_actuator.access\_protocol.mailbox.Mailbox, method), 21

get\_reader\_epubkey() (aliro\_actuator.access\_protocol.reader.ReaderSession, method), 26

get\_reader\_public\_key() (aliro\_actuator.access\_protocol.user\_device.UserSession, method), 34

get\_reader\_public\_key() (aliro\_actuator.trust\_framework.access\_credential.AccessCredential, method), 40

GET\_RESPONSE (aliro\_actuator.access\_protocol.apdu.INS, attribute), 11

get\_signaling\_bitmap() (aliro\_actuator.access\_protocol.user\_device.UserDevice, method), 30

get\_timestamp() (aliro\_actuator.access\_document.Document, method), 8

get\_tlv() (aliro\_actuator.access\_protocol.mailbox.Mailbox, method), 21

get\_tlv() (aliro\_actuator.access\_protocol.tlv.TLV, method), 28

get\_transaction\_type() (aliro\_actuator.access\_protocol.user\_device.UserSession, method), 34

get\_value() (aliro\_actuator.access\_protocol.tlv.TLV, method), 29

get\_x() (aliro\_actuator.trust\_framework.key.PublicKey, method), 43

get\_y() (aliro\_actuator.trust\_framework.key.PublicKey, method), 43

Global (class in aliro\_actuator), 44

## H

handle (aliro\_actuator.hw\_driver.pn7160\_driver.api.nfc\_tag\_info\_t, attribute), 36

handle\_auth0() (aliro\_actuator.access\_protocol.reader.Reader, method), 24

handle\_auth0() (aliro\_actuator.access\_protocol.user\_device.UserDevice, method), 30

handle\_auth1() (aliro\_actuator.access\_protocol.reader.Reader, method), 24

handle\_auth1() (aliro\_actuator.access\_protocol.user\_device.UserDevice, method), 31

handle\_auth1\_credential\_public\_key() (aliro\_actuator.access\_protocol.reader.Reader, method), 25

handle\_control\_flow()

(aliro\_actuator.access\_protocol.reader.Reader, method), 25

handle\_control\_flow() (aliro\_actuator.access\_protocol.user\_device.UserDevice, method), 31

handle\_exchange() (aliro\_actuator.access\_protocol.reader.Reader, method), 25

handle\_exchange() (aliro\_actuator.access\_protocol.user\_device.UserDevice, method), 31

handle\_load\_cert() (aliro\_actuator.access\_protocol.reader.Reader, method), 25

handle\_load\_cert() (aliro\_actuator.access\_protocol.user\_device.UserDevice, method), 31

handle\_select() (aliro\_actuator.access\_protocol.reader.Reader, method), 25

handle\_select() (aliro\_actuator.access\_protocol.user\_device.UserDevice, method), 31

has\_identifier() (aliro\_actuator.trust\_framework.access\_credential.AccessCredential, method), 40

## I

INCORRECT\_P1\_P2 (aliro\_actuator.access\_protocol.apdu.StatusBytes, attribute), 12

INCORRECT\_PARAMETERS\_IN\_DATA

(aliro\_actuator.access\_protocol.apdu.StatusBytes, attribute), 12

INCORRECT\_SECURE\_MESSAGING\_DOS

(aliro\_actuator.access\_protocol.apdu.StatusBytes, attribute), 12

INDEX\_TAG (aliro\_actuator.access\_protocol.mailbox.Mailbox, attribute), 21

initialization() (aliro\_actuator.transport\_protocol.ble\_uwb.BLEUWB, method), 37

initialization() (aliro\_actuator.transport\_protocol.nfc.NFC, method), 38

initialization() (aliro\_actuator.transport\_protocol.socket.Socket, method), 38

initialization() (aliro\_actuator.transport\_protocol.TransportProtocolBase, method), 39

initialize() (*aliro\_actuator.hw\_driver.pn7160\_driver.DIOLOGICAL\_CHANNEL\_NOT\_SUPPORTED*  
     *method*), 37  
 INITIATE\_ACCESS\_PROTOCOL  
     (*aliro\_actuator.transport\_protocol.MessageType* *loglevel\_mapping* (*aliro\_actuator.log.InterceptHandler*  
     *attribute*), 39 *attribute*), 44  
 INS (*class* in *aliro\_actuator.access\_protocol.apdu*), 10  
 InterceptHandler (*class* in *aliro\_actuator.log*), 43  
 INVALID\_CLASS (*aliro\_actuator.access\_protocol.apdu.StatusBytes* *method*), 26  
     *attribute*), 12  
 INVALID\_INSTRUCTION  
     (*aliro\_actuator.access\_protocol.apdu.StatusBytes* *attribute*), 13  
 InvalidAIDError, 19  
 InvalidCLAError, 19  
 InvalidCommandDataError, 19  
 InvalidCommandError, 19  
 InvalidIdentifierError, 41  
 InvalidINSError, 19  
 InvalidKeyError, 41  
 InvalidKeyFormatError, 41  
 InvalidLcError, 19  
 InvalidLeError, 19  
 InvalidModeError, 38  
 InvalidParameterError, 19  
 InvalidResponseDataError, 20  
 InvalidResponseError, 20  
 InvalidStatusError, 20  
 is\_def (*aliro\_actuator.hw\_driver.pn7160\_driver.api.ndef\_info\_t* *attribute*), 35  
 is\_started() (*aliro\_actuator.access\_protocol.user\_device.MailboxSession*  
     *method*), 29  
 is\_writable (*aliro\_actuator.hw\_driver.pn7160\_driver.api.ndef\_info\_t* *attribute*), 35  
     *attribute*), 35  
 IssuerAuth (*class* in *aliro\_actuator.access\_document.access\_credential*),  
     7  
**K**  
 Key (*class* in *aliro\_actuator.trust\_framework.key*), 42  
 KEY\_SLOT (*aliro\_actuator.access\_protocol.apdu.Auth1Response*  
     *attribute*), 9  
 KEY\_SLOT\_LEN (*aliro\_actuator.access\_protocol.defines.Auth1*  
     *attribute*), 15  
 KEY\_SLOT\_TAG (*aliro\_actuator.access\_protocol.defines.Auth1*  
     *attribute*), 15  
 KeyLookupFailed, 41  
 KeyPair (*class* in *aliro\_actuator.trust\_framework.key*),  
     42  
**L**  
 LOAD\_CERT (*aliro\_actuator.access\_protocol.apdu.INS* *attribute*), 11  
 logger (*aliro\_actuator.Global* *attribute*), 44  
 LoggingConfigError, 44

**M**  
 Mailbox (*class* in *aliro\_actuator.access\_protocol.mailbox*),  
     21  
 MAILBOX\_DATA\_TAG (*aliro\_actuator.access\_protocol.defines.Auth1*  
     *attribute*), 15  
 MailboxFormatError, 22  
 MailboxPermissionError, 22  
 MailboxSession (*class* in  
     *aliro\_actuator.access\_protocol.user\_device*),  
     29  
 MailboxSession.Command (*class* in  
     *aliro\_actuator.access\_protocol.user\_device*),  
     29  
 main\_loop() (*aliro\_actuator.access\_protocol.user\_device.UserDevice*  
     *method*), 31  
 MASK\_A (*aliro\_actuator.hw\_driver.pn7160\_driver.api.TECHNOLOGY\_MASK*  
     *attribute*), 35  
 MAX\_COMMAND\_LEN (*aliro\_actuator.access\_protocol.defines.Select*  
     *attribute*), 16  
 MAX\_COMMAND\_TAG (*aliro\_actuator.access\_protocol.defines.Select*  
     *attribute*), 16  
 max\_ndef\_length (*aliro\_actuator.hw\_driver.pn7160\_driver.api.ndef\_info\_t*  
     *attribute*), 35  
 MAX\_RESPONSE\_LEN (*aliro\_actuator.access\_protocol.defines.Select*  
     *attribute*), 16  
 MAX\_RESPONSE\_TAG (*aliro\_actuator.access\_protocol.defines.Select*  
     *attribute*), 16  
 MEMORY\_FAILURE (*aliro\_actuator.access\_protocol.apdu.StatusBytes*  
     *attribute*), 13  
 Message (*class* in *aliro\_actuator.access\_protocol.apdu*),  
     11  
 MessageTooLongError, 20  
 MessageType (*class* in  
     *aliro\_actuator.transport\_protocol*), 39  
 MissingPublicKeyError, 41  
 Mode (*class* in *aliro\_actuator.transport\_protocol*), 39  
 module  
     *aliro\_actuator*, 44  
     *aliro\_actuator.access\_document*, 8  
     *aliro\_actuator.access\_document.access\_credential*,  
         7  
     *aliro\_actuator.access\_document.errors*, 7  
     *aliro\_actuator.access\_document.revocation\_document*,  
         8  
     *aliro\_actuator.access\_protocol*, 35

aliro\_actuator.access\_protocol.apdu, 8  
 aliro\_actuator.access\_protocol.authentication, 13  
 aliro\_actuator.access\_protocol.definitions, 14  
 aliro\_actuator.access\_protocol.encryption, 17  
 aliro\_actuator.access\_protocol.errors, 19  
 aliro\_actuator.access\_protocol.mailbox, 21  
 aliro\_actuator.access\_protocol.reader, 22  
 aliro\_actuator.access\_protocol.tlv, 27  
 aliro\_actuator.access\_protocol.user\_device, 29  
 aliro\_actuator.hw\_driver, 37  
 aliro\_actuator.hw\_driver.pn7160\_driver, 37  
 aliro\_actuator.hw\_driver.pn7160\_driver.api, 35  
 aliro\_actuator.hw\_driver.pn7160\_driver.errors, 36  
 aliro\_actuator.log, 43  
 aliro\_actuator.transport\_protocol, 39  
 aliro\_actuator.transport\_protocol.ble\_uwb, 37  
 aliro\_actuator.transport\_protocol.errors, 38  
 aliro\_actuator.transport\_protocol.nfc, 38  
 aliro\_actuator.transport\_protocol.socket, 38  
 aliro\_actuator.trust\_framework, 43  
 aliro\_actuator.trust\_framework.access\_credential, 40  
 aliro\_actuator.trust\_framework.certificate, 40  
 aliro\_actuator.trust\_framework.errors, 41  
 aliro\_actuator.trust\_framework.key, 42  
 aliro\_actuator.trust\_framework.reader\_identifier, 43

**N**  
 NCLError, 36  
 NCINotFoundError, 36  
 ndef\_info\_t (class in aliro\_actuator.hw\_driver.pn7160\_driver.api), 35  
 NFC (aliro\_actuator.access\_protocol.definitions.TransportProtocol attribute), 16  
 NFC (class in aliro\_actuator.transport\_protocol.nfc), 38  
 nfc\_tag\_info\_t (class in aliro\_actuator.hw\_driver.pn7160\_driver.api), 36  
 nfcHostCardEmulationCallback\_t (class in aliro\_actuator.hw\_driver.pn7160\_driver.api), 35

onTagCallback\_t (class in aliro\_actuator.hw\_driver.pn7160\_driver.api), 36  
 NO\_PRECISE\_DIAGNOSIS (aliro\_actuator.access\_protocol.apdu.StatusBytes attribute), 13  
 NoDataReceivedError, 36, 38  
 NoDeviceConnectedError, 38  
 NONE (aliro\_actuator.access\_protocol.apdu.S2 attribute), 12  
 NoReaderError, 36  
 NoTagError, 36  
 NOTIFY\_TAG (aliro\_actuator.access\_protocol.definitions.Exchange attribute), 15

**O**  
 onDataReceived (aliro\_actuator.hw\_driver.pn7160\_driver.api.nfcHostCardEmulation attribute), 35  
 onHostCardEmulationActivated (aliro\_actuator.hw\_driver.pn7160\_driver.api.nfcHostCardEmulation attribute), 35  
 onHostCardEmulationDeactivated (aliro\_actuator.hw\_driver.pn7160\_driver.api.nfcHostCardEmulation attribute), 35  
 onTagArrival (aliro\_actuator.hw\_driver.pn7160\_driver.api.nfcTagCallback attribute), 36  
 onTagDeparture (aliro\_actuator.hw\_driver.pn7160\_driver.api.nfcTagCallback attribute), 36

**P**  
 parse() (aliro\_actuator.access\_document.access\_credential.AccessDocument method), 7  
 parse\_as\_auth0() (aliro\_actuator.access\_protocol.apdu.Command method), 10  
 parse\_as\_auth0() (aliro\_actuator.access\_protocol.apdu.Response method), 11  
 parse\_as\_auth1() (aliro\_actuator.access\_protocol.apdu.Command method), 10  
 parse\_as\_auth1() (aliro\_actuator.access\_protocol.apdu.Response method), 11  
 parse\_as\_control\_flow() (aliro\_actuator.access\_protocol.apdu.Command method), 10  
 parse\_as\_control\_flow() (aliro\_actuator.access\_protocol.apdu.Response method), 11  
 parse\_as\_envelope() (aliro\_actuator.access\_protocol.apdu.Command method), 10  
 parse\_as\_envelope() (aliro\_actuator.access\_protocol.apdu.Response method), 11

|                                                                                                                    |                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <code>parse_as_exchange()</code><br>( <i>aliro_actuator.access_protocol.apdu.Command</i><br>method), 10            | <code>READER</code> ( <i>aliro_actuator.access_protocol.encryption.DeviceType</i><br>attribute), 17                          |
| <code>parse_as_exchange()</code><br>( <i>aliro_actuator.access_protocol.apdu.Response</i><br>method), 11           | <code>READER</code> ( <i>aliro_actuator.transport_protocol.Mode</i> attribute), 39                                           |
| <code>parse_as_get_response()</code><br>( <i>aliro_actuator.access_protocol.apdu.Command</i><br>method), 10        | <code>Reader</code> (class in <i>aliro_actuator.access_protocol.reader</i> ), 22                                             |
| <code>parse_as_get_response()</code><br>( <i>aliro_actuator.access_protocol.apdu.Response</i><br>method), 11       | <code>READER_EPUBK_LEN</code> ( <i>aliro_actuator.access_protocol.defines.Auth0</i><br>attribute), 14                        |
| <code>parse_as_load_cert()</code><br>( <i>aliro_actuator.access_protocol.apdu.Command</i><br>method), 10           | <code>READER_EPUBK_TAG</code> ( <i>aliro_actuator.access_protocol.defines.Auth0</i><br>attribute), 14                        |
| <code>parse_as_load_cert()</code><br>( <i>aliro_actuator.access_protocol.apdu.Response</i><br>method), 12          | <code>READER_EPUBK_TAG</code> ( <i>aliro_actuator.access_protocol.defines.ReaderAuth</i><br>attribute), 15                   |
| <code>parse_as_select()</code> ( <i>aliro_actuator.access_protocol.apdu.Command</i><br>method), 10                 | <code>READER_EPUBK_TAG</code> ( <i>aliro_actuator.access_protocol.defines.UserDeviceAuth</i><br>attribute), 16               |
| <code>parse_as_select()</code> ( <i>aliro_actuator.access_protocol.apdu.Response</i><br>method), 12                | <code>reader_group_identifier</code><br>( <i>aliro_actuator.access_protocol.reader.Reader</i><br>property), 25               |
| <code>parse_command()</code> ( <i>aliro_actuator.access_protocol.apdu.APDU</i><br>method), 9                       | <code>reader_group_identifier</code><br>( <i>aliro_actuator.access_protocol.reader.ReaderSession</i><br>property), 26        |
| <code>parse_response()</code> ( <i>aliro_actuator.access_protocol.apdu.APDU</i><br>method), 9                      | <code>reader_group_sub_identifier</code><br>( <i>aliro_actuator.access_protocol.user_device.UserSession</i><br>property), 34 |
| <code>PN7160DriverError</code> , 36                                                                                | <code>reader_group_sub_identifier</code><br>( <i>aliro_actuator.access_protocol.reader.Reader</i><br>property), 25           |
| <code>print_to_log()</code> ( <i>aliro_actuator.access_protocol.reader.ReaderFastCacheEntry</i><br>method), 26     | <code>reader_group_sub_identifier</code><br>( <i>aliro_actuator.access_protocol.reader.ReaderSession</i><br>property), 26    |
| <code>PrivateKey</code> (class in <i>aliro_actuator.trust_framework.key</i> ), 42                                  | <code>reader_group_sub_identifier</code><br>( <i>aliro_actuator.access_protocol.user_device.UserSession</i><br>property), 34 |
| <code>PROPRIETARY_TAG</code> ( <i>aliro_actuator.access_protocol.defines.Select</i><br>attribute), 16              | <code>reader_identifier</code> ( <i>aliro_actuator.access_protocol.reader.Reader</i><br>property), 25                        |
| <code>protocol</code> ( <i>aliro_actuator.hw_driver.pn7160_driver.api.nfc_tag_info</i><br>attribute), 36           | <code>reader_identifier</code> ( <i>aliro_actuator.access_protocol.reader.ReaderSession</i><br>property), 27                 |
| <code>PROTOCOL_VERSION_NOT_SUPPORTED</code><br>( <i>aliro_actuator.access_protocol.apdu.S2</i><br>attribute), 12   | <code>reader_identifier</code> ( <i>aliro_actuator.access_protocol.user_device.UserSession</i><br>property), 34              |
| <code>PublicKey</code> (class in <i>aliro_actuator.trust_framework.key</i> ), 43                                   | <code>READER_IDENTIFIER_LEN</code><br>( <i>aliro_actuator.access_protocol.defines.Auth0</i><br>attribute), 14                |
| <b>R</b>                                                                                                           | <code>READER_IDENTIFIER_TAG</code><br>( <i>aliro_actuator.access_protocol.defines.Auth0</i><br>attribute), 14                |
| <code>RE_VENDOR_SPECIFIC_MAX_LEN</code><br>( <i>aliro_actuator.access_protocol.defines.Auth0</i><br>attribute), 14 | <code>READER_IDENTIFIER_TAG</code><br>( <i>aliro_actuator.access_protocol.defines.ReaderAuth</i><br>attribute), 16           |
| <code>RE_VENDOR_SPECIFIC_TAG</code><br>( <i>aliro_actuator.access_protocol.defines.Auth0</i><br>attribute), 14     | <code>READER_IDENTIFIER_TAG</code><br>( <i>aliro_actuator.access_protocol.defines.UserDeviceAuth</i><br>attribute), 16       |
| <code>read()</code> ( <i>aliro_actuator.access_protocol.mailbox.Mailbox</i><br>method), 21                         | <code>READER_SIG_LEN</code> ( <i>aliro_actuator.access_protocol.defines.Auth1</i><br>attribute), 15                          |
| <code>READ_LEN</code> ( <i>aliro_actuator.access_protocol.defines.Exchange</i><br>attribute), 15                   | <code>READER_SIG_TAG</code> ( <i>aliro_actuator.access_protocol.defines.Auth1</i><br>attribute), 15                          |
| <code>READ_TAG</code> ( <i>aliro_actuator.access_protocol.defines.Exchange</i><br>attribute), 15                   |                                                                                                                              |

ReaderAuth (class in aliro\_actuator.access\_protocol.defines, 15  
 ReaderFastCacheEntry (class in aliro\_actuator.access\_protocol.reader), 26  
 ReaderIdentifier (class in aliro\_actuator.trust\_framework.reader\_identifier), 43  
 ReaderSession (class in aliro\_actuator.access\_protocol.reader), 26  
 ReaderStorage (class in aliro\_actuator.access\_protocol.reader), 27  
 receive\_message() (aliro\_actuator.hw\_driver.pn7160\_driver.Driver method), 37  
 remove\_kpersistent() (aliro\_actuator.access\_protocol.reader.ReaderStorage method), 27  
 remove\_kpersistent() (aliro\_actuator.access\_protocol.user\_device.UserStorage method), 34  
 REQUEST (aliro\_actuator.transport\_protocol.MessageType attribute), 39  
 RESPONSE (aliro\_actuator.transport\_protocol.MessageType attribute), 39  
 Response (class in aliro\_actuator.access\_protocol.apdu), 11  
 response\_auth0() (aliro\_actuator.access\_protocol.user\_device.UserSessionState method), 31  
 response\_auth1() (aliro\_actuator.access\_protocol.user\_device.UserSessionState method), 32  
 response\_control\_flow() (aliro\_actuator.access\_protocol.user\_device.UserDevice method), 32  
 response\_envelope() (aliro\_actuator.access\_protocol.user\_device.UserDevice method), 32  
 response\_exchange() (aliro\_actuator.access\_protocol.user\_device.UserDevice method), 32  
 response\_get\_response() (aliro\_actuator.access\_protocol.user\_device.UserDevice method), 32  
 response\_load\_cert() (aliro\_actuator.access\_protocol.user\_device.UserDevice method), 32  
 response\_select() (aliro\_actuator.access\_protocol.user\_device.UserDevice method), 32  
 retrieve() (aliro\_actuator.access\_document.Document method), 8  
 return\_exchange\_error\_and\_close\_channel() (aliro\_actuator.access\_protocol.user\_device.UserDevice method), 32  
 REVOCATION\_TIMESTAMP\_LEN (aliro\_actuator.access\_protocol.defines.Auth1 attribute), 15  
 REVOCATION\_TIMESTAMP\_TAG (aliro\_actuator.access\_protocol.defines.Auth1 attribute), 15  
 RevocationDocument (class in aliro\_actuator.access\_document.revocation\_document), 8  
 S1 (class in aliro\_actuator.access\_protocol.apdu), 12  
 S1\_LEN (aliro\_actuator.access\_protocol.defines.ControlFlow attribute), 15  
 S2 (class in aliro\_actuator.access\_protocol.apdu), 12  
 S2\_LEN (aliro\_actuator.access\_protocol.defines.ControlFlow attribute), 15  
 S2\_TAG (aliro\_actuator.access\_protocol.defines.ControlFlow attribute), 15  
 SECURITY\_STATUS\_NOT\_SATISFIED (aliro\_actuator.access\_protocol.apdu.StatusBytes attribute), 13  
 SELECT (aliro\_actuator.access\_protocol.apdu.INS attribute), 11  
 Select (class in aliro\_actuator.access\_protocol.defines), 16  
 SELECT\_DONE (aliro\_actuator.access\_protocol.user\_device.UserSessionState attribute), 34  
 SELECT\_STOP\_DONE (aliro\_actuator.access\_protocol.user\_device.UserSessionState attribute), 34  
 send\_initiate\_access\_protocol\_notification() (aliro\_actuator.access\_protocol.user\_device.UserDevice method), 32  
 send\_message() (aliro\_actuator.hw\_driver.pn7160\_driver.Driver method), 37  
 send\_message() (aliro\_actuator.transport\_protocol.ble\_uwb.BLEUWB method), 37  
 send\_message() (aliro\_actuator.transport\_protocol.nfc.NFC method), 38  
 send\_message() (aliro\_actuator.transport\_protocol.socket.Socket method), 39  
 send\_message() (aliro\_actuator.transport\_protocol.TransportProtocolBase method), 39  
 SESSION\_START (aliro\_actuator.access\_protocol.user\_device.UserSessionState attribute), 34  
 SessionError, 20  
 SET (aliro\_actuator.access\_protocol.user\_device.MailboxSession.Command attribute), 29  
 set() (aliro\_actuator.access\_protocol.mailbox.Mailbox method), 21  
 set\_access\_credential() (aliro\_actuator.access\_protocol.user\_device.UserSessionState method), 34

set\_auth0\_data() (aliro\_actuator.access\_protocol.user\_device.UserSession, 16  
 method), 34  
 set\_auth1\_info() (aliro\_actuator.access\_protocol.reader.ReaderSession, 16  
 method), 27  
 set\_cert\_and\_verify() (aliro\_actuator.access\_protocol.user\_device.UserSession, 13  
 method), 34  
 set\_credential\_ephemeral\_key() (aliro\_actuator.access\_protocol.reader.ReaderSession, 27  
 method), 27  
 set\_credential\_public\_key() (aliro\_actuator.access\_protocol.reader.ReaderSession, 27  
 method), 27  
 set\_cryptogram\_info() (aliro\_actuator.access\_protocol.reader.ReaderSession, 27  
 method), 27  
 set\_flag() (aliro\_actuator.access\_protocol.reader.ReaderSession, 34  
 method), 27  
 set\_initiate\_access\_protocol\_info() (aliro\_actuator.access\_protocol.reader.ReaderSession, 27  
 method), 27  
 SET\_LEN (aliro\_actuator.access\_protocol.defines.Exchange, 15  
 attribute), 15  
 set\_logger() (aliro\_actuator.Global class method), 44  
 set\_response\_vendor\_extension() (aliro\_actuator.access\_protocol.reader.ReaderSession, 27  
 method), 27  
 set\_select\_info() (aliro\_actuator.access\_protocol.reader.ReaderSession, 27  
 method), 27  
 set\_shared\_key() (aliro\_actuator.access\_protocol.reader.ReaderSession, 27  
 method), 27  
 set\_shared\_key() (aliro\_actuator.access\_protocol.user\_device.UserSession, 34  
 method), 34  
 SET\_TAG (aliro\_actuator.access\_protocol.defines.Exchange, 15  
 attribute), 15  
 setup\_connection() (aliro\_actuator.access\_protocol.reader.ReaderSession, 26  
 method), 26  
 setup\_connection() (aliro\_actuator.access\_protocol.user\_device.UserSession, 33  
 method), 33  
 sign() (aliro\_actuator.trust\_framework.access\_credential.AccessCredential, 40  
 method), 40  
 sign() (aliro\_actuator.trust\_framework.key.KeyPair, 42  
 method), 42  
 sign() (aliro\_actuator.trust\_framework.key.PrivateKey, 42  
 method), 42  
 SIGNALING\_BITMAP\_LEN (aliro\_actuator.access\_protocol.defines.Auth1, 15  
 attribute), 15  
 SIGNALING\_BITMAP\_TAG (aliro\_actuator.access\_protocol.defines.Auth1, 15  
 attribute), 15  
 Socket (class in aliro\_actuator.transport\_protocol.socket), 38  
 SOCKET\_BLE (aliro\_actuator.access\_protocol.defines.TransportProtocol, 16  
 attribute), 16  
 SOCKET\_NFC (aliro\_actuator.access\_protocol.defines.TransportProtocol, 16  
 attribute), 16  
 STANDARD (aliro\_actuator.access\_protocol.apdu.Transaction, 13  
 attribute), 13  
 start() (aliro\_actuator.access\_protocol.user\_device.MailboxSession, 30  
 method), 30  
 start\_new\_session() (aliro\_actuator.access\_protocol.reader.ReaderSession, 26  
 method), 26  
 start\_new\_session() (aliro\_actuator.access\_protocol.user\_device.UserDevice, 33  
 method), 33  
 START\_OF\_DATA\_TAG (aliro\_actuator.access\_protocol.mailbox.Mailbox, 21  
 attribute), 21  
 state\_valid() (aliro\_actuator.access\_protocol.user\_device.UserSession, 34  
 method), 27  
 StatusBytes (class in aliro\_actuator.access\_protocol.apdu), 12  
 step\_up\_aid\_select\_required() (aliro\_actuator.access\_protocol.reader.ReaderSession, 27  
 method), 27  
 step\_up\_transaction() (aliro\_actuator.access\_protocol.reader.ReaderSession, 26  
 method), 26  
 stop() (aliro\_actuator.access\_protocol.user\_device.MailboxSession, 30  
 method), 30  
 stop() (aliro\_actuator.access\_document.Document, 8  
 method), 8  
 SUCCESS (aliro\_actuator.access\_protocol.apdu.StatusBytes, 13  
 attribute), 13  
 support\_extended\_length\_apdu (aliro\_actuator.access\_protocol.Device, 35  
 property), 35  
 sw1 (aliro\_actuator.access\_protocol.apdu.Response, 12  
 property), 12  
 sw2 (aliro\_actuator.access\_protocol.apdu.Response, 12  
 property), 12  
 technology (aliro\_actuator.hw\_driver.pn7160\_driver.api.nfc\_tag\_info\_t, 36  
 attribute), 36  
 TECHNOLOGY\_MASK (class in aliro\_actuator.hw\_driver.pn7160\_driver.api), 35  
 TEST (aliro\_actuator.access\_protocol.user\_device.UserMode, 33  
 attribute), 33  
 TLV (class in aliro\_actuator.access\_protocol.tlv), 27  
 TlvError, 29  
 to\_bytes() (aliro\_actuator.access\_protocol.apdu.Message, 11  
 method), 11  
 to\_bytes() (aliro\_actuator.access\_protocol.tlv.TLV, 29  
 method), 29  
 SOCKET\_BLE (aliro\_actuator.access\_protocol.defines.TransportProtocol, 16  
 attribute), 16

[to\\_data\(\)](#) ([aliro\\_actuator.access\\_protocol.tlv.TLV](#) method), 29  
[to\\_print\(\)](#) ([aliro\\_actuator.access\\_protocol.tlv.TLV](#) method), 29  
[Transaction](#) (class in [aliro\\_actuator.access\\_protocol.apdu](#)), 13  
[TRANSACTION\\_CODE\\_LEN](#) ([aliro\\_actuator.access\\_protocol.defines.Auth0](#) attribute), 14  
[TRANSACTION\\_CODE\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.Auth0](#) attribute), 14  
[TRANSACTION\\_ID\\_LEN](#) ([aliro\\_actuator.access\\_protocol.defines.Auth0](#) attribute), 14  
[TRANSACTION\\_ID\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.Auth0](#) attribute), 14  
[transaction\\_identifier](#) ([aliro\\_actuator.access\\_protocol.reader.ReaderSession](#) property), 27  
[TRANSACTION\\_IDENTIFIER\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.ReaderAuth](#) attribute), 16  
[TRANSACTION\\_IDENTIFIER\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.UserDeviceAuth](#) attribute), 17  
[transaction\\_initiation\(\)](#) ([aliro\\_actuator.access\\_protocol.reader.Reader](#) method), 26  
[transaction\\_initiation\(\)](#) ([aliro\\_actuator.access\\_protocol.user\\_device.UserSession](#) method), 33  
[transaction\\_termination\(\)](#) ([aliro\\_actuator.access\\_protocol.reader.Reader](#) method), 26  
[transaction\\_termination\(\)](#) ([aliro\\_actuator.access\\_protocol.user\\_device.UserSession](#) method), 33  
[TransactionCode](#) (class in [aliro\\_actuator.access\\_protocol.apdu](#)), 13  
[TransportProtocol](#) (class in [aliro\\_actuator.access\\_protocol.defines](#)), 16  
[TransportProtocolBase](#) (class in [aliro\\_actuator.transport\\_protocol](#)), 39  
[TransportProtocolError](#), 38  
[TrustFrameworkError](#), 41  
[TYPE\\_LEN](#) ([aliro\\_actuator.access\\_protocol.defines.Select](#) attribute), 16  
[TYPE\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.Select](#) attribute), 16  
**U**  
[uid](#) ([aliro\\_actuator.hw\\_driver.pn7160\\_driver.api.nfc\\_tag\\_info\\_t](#) attribute), 36  
[uid\\_length](#) ([aliro\\_actuator.hw\\_driver.pn7160\\_driver.api.nfc\\_tag\\_info\\_t](#) attribute), 36  
[UnexpectedCommandError](#), 20  
[UnexpectedMessageTypeError](#), 38  
[UnexpectedNotificationDataError](#), 20  
[UnexpectedResponseError](#), 20  
[UnknownVersionRequestedError](#), 38  
[UPDATE\\_DOC\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.Exchange](#) attribute), 15  
[update\\_state\(\)](#) ([aliro\\_actuator.access\\_protocol.user\\_device.UserSession](#) method), 34  
[URSK\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.Exchange](#) attribute), 15  
[USAGE](#) ([aliro\\_actuator.access\\_protocol.defines.ReaderAuth](#) attribute), 16  
[USAGE](#) ([aliro\\_actuator.access\\_protocol.defines.UserDeviceAuth](#) attribute), 17  
[USAGE\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.ReaderAuth](#) attribute), 16  
[USAGE\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.UserDeviceAuth](#) attribute), 17  
[USER](#) ([aliro\\_actuator.access\\_protocol.encryption.DeviceType](#) attribute), 17  
[USER](#) ([aliro\\_actuator.access\\_protocol.user\\_device.UserMode](#) attribute), 33  
[USER\\_DEVICE](#) ([aliro\\_actuator.access\\_protocol.apdu.TransactionCode](#) attribute), 13  
[USER\\_DEVICE](#) ([aliro\\_actuator.transport\\_protocol.Mode](#) attribute), 39  
[USER\\_DEVICE\\_SECURE\\_ACTION](#) ([aliro\\_actuator.access\\_protocol.apdu.TransactionCode](#) attribute), 13  
[USER\\_DEVICE\\_SIG\\_LEN](#) ([aliro\\_actuator.access\\_protocol.defines.Auth1](#) attribute), 15  
[USER\\_DEVICE\\_SIG\\_TAG](#) ([aliro\\_actuator.access\\_protocol.defines.Auth1](#) attribute), 15  
[UserDevice](#) (class in [aliro\\_actuator.access\\_protocol.user\\_device](#)), 30  
[UserDeviceAuth](#) (class in [aliro\\_actuator.access\\_protocol.defines](#)), 16  
[UserMode](#) (class in [aliro\\_actuator.access\\_protocol.user\\_device](#)), 33  
[UserSession](#) (class in [aliro\\_actuator.access\\_protocol.user\\_device](#)), 33  
[UserSessionState](#) (class in [aliro\\_actuator.access\\_protocol.user\\_device](#)), 34  
[UserStorage](#) (class in [aliro\\_actuator.access\\_protocol.user\\_device](#)), 34

## V

`validate_credential()` (*aliro\_actuator.access\_document.access\_credential.AccessDocument* method), 7

`VENDOR_SPECIFIC_MAX_LEN` (*aliro\_actuator.access\_protocol.defines.Auth0* attribute), 14

`VENDOR_SPECIFIC_TAG` (*aliro\_actuator.access\_protocol.defines.Auth0* attribute), 14

`VENDOR_SPECIFIC_TAG` (*aliro\_actuator.access\_protocol.defines.Select* attribute), 16

`VerificationError`, 18

`verify()` (*aliro\_actuator.trust\_framework.certificate.Certificate* method), 41

`verify()` (*aliro\_actuator.trust\_framework.key.KeyPair* method), 42

`verify()` (*aliro\_actuator.trust\_framework.key.PublicKey* method), 43

`VersionError`, 20

## W

`wait_for_command()` (*aliro\_actuator.access\_protocol.user\_device.UserDevice* method), 33

`wait_for_connection()` (*aliro\_actuator.transport\_protocol.ble\_uwb.BLEUWB* method), 37

`wait_for_connection()` (*aliro\_actuator.transport\_protocol.nfc.NFC* method), 38

`wait_for_connection()` (*aliro\_actuator.transport\_protocol.socket.Socket* method), 39

`wait_for_connection()` (*aliro\_actuator.transport\_protocol.TransportProtocolBase* method), 39

`wait_for_initiate_access_protocol_notification()` (*aliro\_actuator.access\_protocol.reader.Reader* method), 26

`wait_for_reader()` (*aliro\_actuator.hw\_driver.pn7160\_driver.Driver* method), 37

`wait_for_tag()` (*aliro\_actuator.hw\_driver.pn7160\_driver.Driver* method), 37

`WRITE` (*aliro\_actuator.access\_protocol.user\_device.MailboxSession.Command* attribute), 29

`write()` (*aliro\_actuator.access\_protocol.mailbox.Mailbox* method), 22

`WRITE_TAG` (*aliro\_actuator.access\_protocol.defines.Exchange* attribute), 15

`WRONG_LENGTH_IN_LC` (*aliro\_actuator.access\_protocol.apdu.StatusBytes* attribute), 13